

// BKLN SOFTWARE & SYSTEMS · SERIE DE APRENDIZAJE

IA y Machine Learning *con Python.*

// serie en tres libros · ed. 01 · 2026

LIBRO I – FUNDAMENTOS

01. ¿Qué es la IA?	20 min
02. NumPy – vectores y ma...	30 min
03. Pandas – datos en tab...	35 min
04. Matplotlib – ver para...	30 min
05. Regresión lineal	40 min
06. Clasificación – K-Near...	35 min
07. Árboles de decisión y ...	40 min
08. Evaluación de modelos	35 min

LIBRO II – INTERMEDIO

01. Preprocesamiento de d...	40 min
02. Regresión con regular...	35 min
03. Máquinas de soporte v...	40 min
04. Clustering sin etique...	35 min
05. PCA y reducción de di...	35 min
06. Selección de caracter...	30 min
07. Pipelines y automatiz...	40 min
08. Proyecto: sistema de r...	50 min

LIBRO III – DEEP LEARNING

01. Redes neuronales arti...	45
	min
02. Backpropagation y gra...	45
	min
03. Redes convolucionales...	50
	min
04. Redes recurrentes (R...	50
	min
05. Transformers y atenc...	55
	min
06. Fine-tuning de modelos...	60
	min
07. Agentes y herramient...	55
	min
08. Proyecto final – asist...	90
	min

HELLO@BKLNSOFTWARE.COM ·
BKLNSOFTWARE.TECH

+240 222 798 MALABO · GUINEA
086 ECUATORIAL

// ANTES DE EMPEZAR

Nota al lector

Cómo aprovechar este libro en su versión impresa.

Este libro fue diseñado como una experiencia interactiva en web: los ejercicios incluyen un entorno de ejecución de Python en el navegador, los quizzes son interactivos y el progreso se guarda automáticamente.

En esta versión PDF encontrarás el contenido completo — explicaciones, código de todos los ejercicios y las respuestas correctas de cada quiz — listo para leer offline en cualquier dispositivo.

Para la experiencia completa con código ejecutable, visita:

bklsoftware.tech/cursos

// versión web: bklsoftware.tech · contacto: hello@bklsoftware.com · +240 222 798 086



Fundamentos

Datos, NumPy, Pandas y tus primeros modelos

- 01. ¿Qué es la IA? · 20 min
- 02. NumPy – vectores y matrices · 30 min
- 03. Pandas – datos en tablas · 35 min
- 04. Matplotlib – ver para entender · 30 min
- 05. Regresión lineal · 40 min
- 06. Clasificación – K-Nearest Neighbors · 35 min
- 07. Árboles de decisión y Random Forest · 40 min
- 08. Evaluación de modelos · 35 min

// LIBRO I — FUNDAMENTOS · MÓDULO 01 · 20 MIN DE LECTURA

¿Qué es la IA?

Diferencia entre inteligencia artificial, machine learning y deep learning — y por qué importa.

La **inteligencia artificial** (IA) es un campo de la informática cuyo objetivo es construir sistemas capaces de realizar tareas que normalmente requieren inteligencia humana: reconocer imágenes, entender texto, tomar decisiones o predecir resultados. No es ciencia ficción ni magia — es matemática, datos y código bien organizado.

Dentro de la IA existen varias ramas. Las tres más importantes hoy son:

- **Inteligencia Artificial** — el campo completo: cualquier técnica que hace a una máquina "inteligente".
- **Machine Learning (ML)** — una rama de la IA donde los sistemas *aprenden de datos* en lugar de seguir reglas escritas a mano.
- **Deep Learning (DL)** — una rama del ML que usa redes neuronales con muchas capas. Es lo que hay detrás de ChatGPT, DALL-E y los coches autónomos.



LA METÁFORA CLAVE

La IA tradicional es como darle a alguien un **manual de reglas** para jugar ajedrez. El ML es como dejarle ver **un millón de partidas** y que descubra las reglas él solo.

¿Cómo aprende una máquina?

El proceso de machine learning sigue siempre la misma estructura, independientemente del problema:

1. **Datos** — el combustible. Sin datos no hay aprendizaje.
2. **Algoritmo** — el método matemático que extrae patrones de los datos.
3. **Modelo** — el resultado del entrenamiento. Un modelo es una función que transforma entradas en predicciones.
4. **Evaluación** — ¿cómo de buenas son las predicciones? Hay que medirlo.
5. **Predicción** — el modelo en producción, respondiendo preguntas sobre datos nuevos.

Tipos de aprendizaje

Según cómo se usan los datos, el ML se clasifica en tres tipos:

Aprendizaje supervisado — los datos tienen etiquetas. Cada ejemplo tiene una respuesta correcta conocida. Ejemplo: fotos de gatos y perros, donde ya sabes cuál es cuál. El modelo aprende a clasificar imágenes nuevas. La mayoría del ML práctico es supervisado.

Aprendizaje no supervisado — los datos no tienen etiquetas. El modelo busca estructuras ocultas por sí solo. Ejemplo: agrupar clientes por comportamiento de compra sin saber de antemano cuántos grupos existen.

Aprendizaje por refuerzo — el modelo aprende por ensayo y error, recibiendo recompensas o penalizaciones. Es lo que usan los agentes de videojuegos y los robots que aprenden a caminar.

#	tipo	datos	ejemplo	algoritmos
0	Supervisado	Etiquetados	Predecir precio de casa	Regresión, SVM, Árboles
1	No supervisado	Sin etiquetas	Agrupar clientes	K-Means, PCA, DBSCAN
2	Por refuerzo	Recompensas	Jugar al ajedrez	Q-Learning, PPO

Los tres paradigmas del machine learning

Aplicaciones reales

El ML no es teoría académica. Está en todas partes, incluyendo contextos muy cercanos a nosotros: los sistemas de predicción de lluvias para la agricultura, la detección de fraude en transferencias bancarias, los chatbots de atención al cliente, los algoritmos de recomendación de contenido. En Guinea Ecuatorial, sectores como el petróleo, la pesca y la banca ya están explorando estas tecnologías.

El ecosistema Python para ML

Python domina el campo del ML. Estas son las librerías que usarás en este libro:

#	librería	para	módulo
0	NumPy	Álgebra lineal y arrays	<code>import numpy as np</code>
1	Pandas	Tablas de datos (DataFrames)	<code>import pandas as pd</code>
2	Matplotlib	Visualización y gráficos	<code>import matplotlib.pyplot as plt</code>
3	scikit-learn	Algoritmos de ML	<code>from sklearn import ...</code>

El stack básico de ML en Python – los cuatro pilares

PYTHON

```
# El pipeline más simple del mundo: predecir con sklearn
import numpy as np
from sklearn.linear_model import LinearRegression

# Datos: horas de estudio → nota del examen
horas = np.array([[1], [2], [3], [4], [5], [6], [7], [8]])
notas = np.array([40, 52, 61, 70, 75, 82, 88, 94])

# Crear y entrenar el modelo
modelo = LinearRegression()
modelo.fit(horas, notas)

# Predecir: ¿qué nota para 9 horas de estudio?
prediccion = modelo.predict([[9]])
print(f"Predicción para 9 horas: {prediccion[0]:.1f} puntos")
```

I NO TE PREOCUPES SI NO ENTIENDES TODO AÚN

Este ejemplo es un adelanto de lo que aprenderás en el Módulo 5.

Por ahora, observa la estructura: datos → `fit()` → `predict()`. Esa es la base de todo sklearn.

◆ QUIZ RÁPIDO

¿Cuál es la diferencia principal entre ML y la programación tradicional?

A El ML usa Python y la programación tradicional usa Java

B En ML el sistema aprende patrones de datos; en programación tradicional el programador escribe las reglas ✓

C El ML solo funciona para imágenes y la programación tradicional para texto

D No hay diferencia real, son lo mismo con distinto nombre

Respuesta — B. En programación tradicional escribes reglas explícitas (if precio > X, entonces...). En ML le das datos al algoritmo y él descubre las reglas por sí solo.

◆ QUIZ RÁPIDO

Si tienes fotos de tumores etiquetadas como 'maligno' o 'benigno' y entrenas un modelo para clasificar nuevas fotos, ¿qué tipo de aprendizaje es?

A No supervisado, porque las fotos vienen sin contexto

B Supervisado, porque tienes ejemplos con respuestas correctas conocidas ✓

C Por refuerzo, porque el modelo recibe recompensas

D Deep learning, porque las imágenes son complejas

Respuesta — B. Las etiquetas ('maligno'/'benigno') son exactamente eso: respuestas correctas conocidas. Eso define el aprendizaje supervisado.

EJERCICIO 1

· fácil

Explora el ecosistema ML

Importa las cuatro librerías fundamentales del ML en Python y muestra su versión. Este será el "hola mundo" de todos tus proyectos de ML.

PYTHON

```
# Importa las cuatro librerías del stack ML  
# y muestra su versión para confirmar que están disponibles  
import numpy as np  
import pandas as pd  
import matplotlib  
import sklearn  
  
print("NumPy:", np.__version__)  
print("Pandas:", pd.__version__)  
print("Matplotlib:", matplotlib.__version__)  
print("scikit-learn:", sklearn.__version__)  
print()  
print("¡El entorno ML está listo!")
```

Pista: Las versiones se acceden con `__version__` en cada librería.

EJERCICIO 2

· fácil

Tu primera predicción

Tienes datos históricos de temperatura y ventas de bebidas. Entrena un modelo de regresión lineal y predice las ventas para 32°C y 40°C.

PYTHON

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Datos: temperatura (°C) → ventas de bebidas
temperatura = np.array([[20], [25], [28], [30], [33], [35],
ventas      = np.array([150, 180, 200, 220, 260, 290,

# 1. Crea el modelo
modelo = LinearRegression()

# 2. Entrénalo con los datos
modelo.fit(temperatura, ventas)

# 3. Predice las ventas para 32°C y 40°C
pred_32 = modelo.predict([[32]])
pred_40 = modelo.predict([[40]])

print(f"Ventas esperadas a 32°C: {pred_32[0]:.0f} unidades"
print(f"Ventas esperadas a 40°C: {pred_40[0]:.0f} unidades"
```

Pista: El flujo es siempre: crear modelo → fit(X, y) → predict([[valor]])

**MÓDULO COMPLETADO**

Ahora entiendes la diferencia entre IA, ML y DL, los tres tipos de aprendizaje, y has ejecutado tu primera predicción real. En el siguiente módulo aprenderás NumPy — la base matemática sobre la que se construye todo el ecosistema ML.

// LIBRO I — FUNDAMENTOS · MÓDULO 02 · 30 MIN · EJERCICIOS INCLUIDOS

NumPy.

*Vectores, matrices y operaciones matemáticas a escala —
la columna vertebral del ML.*

Imagina que tienes 100.000 precios de casas y necesitas calcular la media, multiplicar todos por un factor de conversión de moneda y encontrar los que están por encima del umbral. Con listas de Python puras, eso requiere bucles. Con **NumPy**, es una sola línea.

NumPy introduce el **array** — una estructura de datos homogénea y multidimensional que opera a velocidad nativa (C por debajo). Todo el ecosistema ML de Python — Pandas, scikit-learn, TensorFlow, PyTorch — usa arrays de NumPy internamente.

Crear arrays

PYTHON

```
import numpy as np

# Desde una lista de Python
a = np.array([10, 20, 30, 40, 50])
print(a)          # [10 20 30 40 50]
print(a.dtype)    # int64
print(a.shape)    # (5,) ← 5 elementos, 1 dimensión

# Secuencias
rango = np.arange(0, 10, 2)    # [0 2 4 6 8]
linspace = np.linspace(0, 1, 5) # [0.  0.25 0.5  0.75 1.  ]

# Arrays especiales
ceros = np.zeros(4)           # [0. 0. 0. 0.]
unos  = np.ones((2, 3))       # matriz 2x3 de unos
aleatorio = np.random.rand(3, 3) # matriz 3x3 aleatoria
```

Arrays 2D: las matrices

En ML, los datos normalmente se representan como matrices donde cada **fila es un ejemplo** cada **columna es una característica** (feature). Un dataset de 1000 casas con 5 características sería una matriz de forma `(1000, 5)`.

PYTHON

```
import numpy as np

# Dataset: 4 casas con 3 características (m², habitaciones, año)
X = np.array([
    [85, 3, 2010],
    [120, 4, 2015],
    [60, 2, 2008],
    [200, 5, 2020],
])

print("Forma:", X.shape)      # (4, 3) - 4 filas, 3 columnas
print("Primera casa:", X[0])  # [85 3 2010]
print("Todos los m²:", X[:, 0]) # [85 120 60 200]
```

Operaciones vectorizadas

La ventaja clave de NumPy es que las operaciones se aplican a *todos los elementos a la vez* sin escribir bucles. Esto es lo que se llama **vectorización**.

PYTHON

```
import numpy as np

precios_fcfa = np.array([15_000_000, 22_000_000, 8_500_000, 35_000_000])

# Convertir a euros (tasa aproximada: 1 EUR ≈ 655 FCFA)
precios_eur = precios_fcfa / 655
print(precios_eur.astype(int)) # [22900 33587 12977 53435]

# Aplicar descuento del 10% a todos
con_descuento = precios_fcfa * 0.90
print(con_descuento / 1_000_000, "millones FCFA")

# Filtrar: ¿cuáles cuestan más de 20M FCFA?
caros = precios_fcfa[precios_fcfa > 20_000_000]
print("Propiedades caras:", caros)
```

Estadísticas básicas

PYTHON

```
import numpy as np

notas = np.array([72, 85, 61, 90, 78, 55, 88, 94, 66, 73])

print("Media:", np.mean(notas)) # 76.2
print("Mediana:", np.median(notas)) # 75.5
print("Desv. estándar:", np.std(notas)) # 12.6
print("Mínimo:", np.min(notas)) # 55
print("Máximo:", np.max(notas)) # 94
print("25%:", np.percentile(notas, 25)) # 66.75
print("75%:", np.percentile(notas, 75)) # 88.5
```

Álgebra lineal – el corazón del ML

Los algoritmos de ML son matemáticamente operaciones de álgebra lineal. NumPy las hace simples:

PYTHON

```
import numpy as np

# Producto punto (dot product) – clave en regresión y redes neuronales
pesos = np.array([0.3, 0.5, 0.2])
features = np.array([100, 3, 2015])
prediccion = np.dot(pesos, features)
print("Predicción:", prediccion)

# Multiplicación de matrices
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = A @ B    # operador @ para matrices en Python 3
print(C)     # [[19 22] [43 50]]

# Transponer
print(A.T)   # [[1 3] [2 4]]
```

★ SHAPE ES TODO

Antes de operar con arrays, mira siempre `array.shape`. La mayoría de los errores en ML provienen de shapes incompatibles. Un error como *shapes (100, 5) and (4,) not aligned* significa que estás multiplicando matrices de dimensiones incorrectas.

◆ QUIZ RÁPIDO

Tienes un dataset con 500 muestras y 8 características. ¿Cuál es el shape del array?

A (8, 500)

B (500, 8) ✓

C (500,)

D (8,)

Respuesta — B. La convención en ML es (n_muestras, n_características).
500 filas (ejemplos) × 8 columnas (features) → shape (500, 8).

◆ QUIZ RÁPIDO

¿Cuál es la diferencia entre `np.zeros(5)` y `np.zeros((5, 5))`?

A Son idénticos, solo es una cuestión de estilo

B El primero crea un vector de 5 elementos; el segundo una matriz 5×5 ✓

C El primero crea enteros; el segundo crea floats

D `np.zeros((5,5))` es una forma incorrecta de escribirlo

Respuesta — B. `np.zeros(5)` → shape (5,) — un vector 1D de 5 ceros.
`np.zeros((5,5))` → shape (5,5) — una matriz 2D de 5×5 ceros. La tupla define las dimensiones.

EJERCICIO 1

· fácil

Estadísticas de ventas con NumPy

Analiza las ventas mensuales de un negocio usando las funciones estadísticas de NumPy.

PYTHON

```
import numpy as np

# Ventas mensuales de un negocio (en miles de FCFA)
ventas = np.array([
    450, 520, 380, 610, 490, 720, 680, 530, 450, 590, 640,
])

# Calcula y muestra:
# 1. Total anual
# 2. Promedio mensual
# 3. Mes con más ventas (índice)
# 4. Meses por encima del promedio (cuántos)
# 5. Crecimiento: último mes vs primer mes (%)

total = np.sum(ventas)
promedio = np.mean(ventas)
mejor_mes = np.argmax(ventas) + 1 # +1 para que sea mes 1-
encima = np.sum(ventas > promedio)
crecimiento = (ventas[-1] - ventas[0]) / ventas[0] * 100

print(f"Total anual: {total:,.} k FCFA")
print(f"Promedio mensual: {promedio:.0f} k FCFA")
print(f"Mejor mes: mes {mejor_mes} ({ventas[mejor_mes-1]:,.})")
print(f"Meses sobre el promedio: {encima}")
print(f"Crecimiento anual: {crecimiento:+.1f}%")
```

Pista: `np.argmax()` devuelve el índice del valor máximo.
`np.sum(condición)` cuenta cuántos True hay.

EJERCICIO 2

· intermedio

Normalización — preparar datos para ML

En ML es fundamental que todas las características estén en rangos similares antes de entrenar. Implementa la normalización min-max para llevar los datos al rango $[0, 1]$.

PYTHON

```
import numpy as np

# Los modelos ML funcionan mejor cuando los datos
# están en rangos similares. La normalización min-max
# lleva todos los valores al rango [0, 1].

edades = np.array([22, 35, 28, 45, 19, 52, 33, 41, 27, 38])

# Fórmula: (x - min) / (max - min)
minimo = np.min(edades)
maximo = np.max(edades)
edades_norm = (edades - minimo) / (maximo - minimo)

print("Originales:", edades)
print("Normalizadas:", edades_norm.round(3))
print("¿Rango [0,1]?", edades_norm.min(), "-", edades_norm.max())

# Ahora normaliza también estos precios de casas (m²):
metros = np.array([55, 80, 120, 65, 200, 90, 45, 150])
metros_norm = (metros - metros.min()) / (metros.max() - metros.min())
print("\nMetros norm.:", metros_norm.round(3))
```

Pista: La fórmula min-max es: $(x - x.min()) / (x.max() - x.min())$.
NumPy aplica esto element-wise automáticamente.

**MÓDULO COMPLETADO**

Dominas los arrays de NumPy: creación, indexación, operaciones vectorizadas y álgebra lineal básica. En el siguiente módulo aprenderás Pandas — construido sobre NumPy — para trabajar con datos reales en formato tabular.

// LIBRO I — FUNDAMENTOS · MÓDULO 03 · 35 MIN · EJERCICIOS INCLUIDOS

Pandas.

DataFrames, exploración y limpieza de datos — donde el 80% del trabajo real de ML ocurre.

En proyectos reales de machine learning, el 80% del tiempo se dedica a entender, limpiar y preparar los datos. **Pandas** es la herramienta que hace posible ese trabajo. Si NumPy son los cimientos, Pandas es la planta baja donde realmente vives.

La estructura central de Pandas es el **DataFrame**: una tabla con filas y columnas donde cada columna puede tener un tipo distinto (números, texto, fechas). Es como una hoja de cálculo, pero controlada desde código.

Crear DataFrames

PYTHON

```
import pandas as pd

# Desde un diccionario (la forma más común)
datos = {
    'nombre':      ['Ana', 'Carlos', 'Fatima', 'Jean', 'María'],
    'edad':         [28, 34, 22, 45, 31],
    'ciudad':       ['Malabo', 'Bata', 'Malabo', 'Ebebiyin', 'Bata'],
    'salario':      [850_000, 1_200_000, 650_000, 980_000, 1_100_000],
    'activo':       [True, True, True, False, True],
}

df = pd.DataFrame(datos)
print(df)
print("\nForma:", df.shape)  # (5, 5) - 5 filas, 5 columnas
```

Exploración inicial

Cada vez que recibes un nuevo dataset, estos son los primeros comandos que ejecutas:

PYTHON

```
import pandas as pd
import numpy as np

# Simular dataset de empleados
np.random.seed(42)
df = pd.DataFrame({
    'edad':      np.random.randint(22, 60, 200),
    'salario':   np.random.normal(950_000, 200_000, 200).astype(int),
    'años_exp':  np.random.randint(0, 20, 200),
    'ciudad':    np.random.choice(['Malabo', 'Bata', 'Mongomo'], 200)
})

print(df.head())          # primeras 5 filas
print("\n", df.info())    # tipos de datos y nulos
print("\n", df.describe()) # estadísticas descriptivas
```

Selección de datos

PYTHON

```
import pandas as pd

df = pd.DataFrame({
    'producto': ['arroz', 'aceite', 'azúcar', 'sal', 'leche'],
    'precio':    [1200, 2500, 800, 400, 1800],
    'stock':     [50, 30, 80, 120, 15],
    'categoría': ['granos', 'aceites', 'granos', 'condimentos', 'lácteos']
})

# Seleccionar columna
print(df['precio'])
print(df[['producto', 'precio']]) # varias columnas

# Filtrar filas (boolean indexing)
baratos = df[df['precio'] < 1000]
print(baratos)

# Filtro múltiple: precio < 2000 Y stock > 20
seleccion = df[(df['precio'] < 2000) & (df['stock'] > 20)]
print(seleccion)

# Seleccionar por posición
print(df.iloc[0])    # primera fila
print(df.iloc[1:3])  # filas 1 y 2
```

Operaciones sobre columnas

PYTHON

```
import pandas as pd

df = pd.DataFrame({
    'producto': ['arroz', 'aceite', 'azúcar', 'sal', 'leche'],
    'precio': [1200, 2500, 800, 400, 1800],
    'stock': [50, 30, 80, 120, 15],
})

# Nueva columna: valor total en almacén
df['valor_total'] = df['precio'] * df['stock']

# Transformar columna existente
df['precio_con_iva'] = df['precio'] * 1.15

# Ordenar
df_ordenado = df.sort_values('valor_total', ascending=False)
print(df_ordenado.to_string(index=False))
```

Agrupación – groupby

`groupby` es una de las operaciones más potentes de Pandas. Agrupa filas por una columna y calcula estadísticas por grupo:

PYTHON

```
import pandas as pd
import numpy as np

np.random.seed(0)
ventas = pd.DataFrame({
    'ciudad': np.random.choice(['Malabo', 'Bata', 'Mongomo'], 100),
    'producto': np.random.choice(['A', 'B', 'C'], 100),
    'monto': np.random.randint(10_000, 500_000, 100),
})

# Ventas totales por ciudad
por_ciudad = ventas.groupby('ciudad')['monto'].sum().sort_values(ascending=True)
print(por_ciudad)

# Media y total por ciudad y producto
resumen = ventas.groupby(['ciudad', 'producto'])['monto'].agg(['mean', 'sum'])
print(resumen)
```

Valores nulos — el problema más común en ML

Los datos reales casi siempre tienen valores faltantes (`NaN`). Los modelos de ML no pueden trabajar con ellos directamente — hay que tratarlos primero.

PYTHON

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'edad':      [25, np.nan, 32, 28, np.nan, 41],
    'salario':    [800_000, 1_200_000, np.nan, 950_000, 1_100_000, 1_300_000],
    'ciudad':     ['Malabo', 'Bata', None, 'Malabo', 'Bata', 'Mongomo']
})

print("Nulos por columna:")
print(df.isnull().sum())

# Estrategia 1: eliminar filas con nulos
df_limpio = df.dropna()

# Estrategia 2: rellenar con la media (para números)
df['edad'].fillna(df['edad'].mean(), inplace=True)
df['salario'].fillna(df['salario'].median(), inplace=True)

# Estrategia 3: rellenar con el valor más frecuente (para texto)
df['ciudad'].fillna(df['ciudad'].mode()[0], inplace=True)

print("\nDespués de limpiar:")
print(df.isnull().sum())
```

 NUNCA ELIMINES NULOS SIN PENSAR

Eliminar filas con nulos puede parecer la solución fácil, pero si el 30% de tus datos tiene valores faltantes y los eliminas, pierdes información valiosa. Rellenar con la media o mediana suele ser mejor. En el Libro 2 aprenderás técnicas más sofisticadas de imputación.

◆ QUIZ RÁPIDO

Tienes un DataFrame con 1000 filas. `df[df['precio'] > 500]` devuelve 342 filas. ¿Qué significa?

- A** 342 productos tienen precio mayor que 500 ✓
- B El precio promedio es 342
- C Hay 342 errores en el DataFrame
- D Faltan 342 valores en la columna precio

Respuesta — A. El boolean indexing `df[condición]` filtra y devuelve solo las filas donde la condición es True. Si devuelve 342 filas, es que 342 productos cumplen precio > 500.

◆ QUIZ RÁPIDO

¿Para qué sirve `df.describe()`?

- A Muestra los primeros 5 registros del DataFrame
- B Muestra estadísticas descriptivas: media, std, min, max, percentiles de columnas numéricas ✓**
- C Describe los nombres de las columnas y sus tipos
- D Calcula correlaciones entre todas las columnas

Respuesta — B. `describe()` genera un resumen estadístico: count, mean, std, min, 25%, 50%, 75%, max para cada columna numérica. Es el primer comando de exploración de cualquier dataset.

EJERCICIO 1

· fácil

Análisis de ventas de una empresa

Analiza un dataset de ventas: explora la estructura, calcula estadísticas por ciudad y producto.

PYTHON

```
import pandas as pd
import numpy as np

# Dataset de ventas
np.random.seed(123)
n = 200
df = pd.DataFrame({
    'fecha':      pd.date_range('2024-01-01', periods=n, fre
    'ciudad':     np.random.choice(['Malabo', 'Bata', 'Mongo
    'producto':   np.random.choice(['Electrónica', 'Ropa', '
    'monto':      np.random.randint(5_000, 200_000, n),
    'unidades':   np.random.randint(1, 50, n),
})

# 1. Muestra las primeras 5 filas
print("≡≡≡ Primeras filas ≡≡≡")
print(df.head())

# 2. Estadísticas básicas del monto
print("\n≡≡≡ Estadísticas de ventas ≡≡≡")
print(df['monto'].describe())

# 3. Ciudad con más ventas en total
print("\n≡≡≡ Ventas por ciudad ≡≡≡")
print(df.groupby('ciudad')['monto'].sum().sort_values(ascen

# 4. Producto más vendido (por unidades)
print("\n≡≡≡ Unidades por producto ≡≡≡")
print(df.groupby('producto')['unidades'].sum().sort_values(
```

Pista: Usa `groupby("columna")["otra"].sum()` para agregar.

EJERCICIO 2

· intermedio

Limpieza de datos reales

Limpia un dataset con problemas reales: valores nulos, tipos incorrectos, duplicados y capitalización inconsistente.

PYTHON

```
import pandas as pd
import numpy as np

# Dataset con problemas típicos: nulos, duplicados, tipos i
data = {
    'nombre':    ['Ana García', 'Carlos López', None, 'Ana G
    'edad':      [28, None, 35, 28, 'treinta', 42],
    'salario':   [850_000, 1_200_000, 780_000, 850_000, None
    'ciudad':    ['Malabo', 'Bata', 'malabo', None, 'Bata',
}
df = pd.DataFrame(data)

print("≡ DataFrame original ≡")
print(df)
print("\nNulos:", df.isnull().sum().sum(), "valores nulos")
print("Duplicados:", df.duplicated().sum(), "filas duplicad

# TU TURNO: limpia el dataset
# 1. Elimina duplicados
df = df.drop_duplicates()

# 2. Convierte 'edad' a numérico (errors='coerce' convierte
df['edad'] = pd.to_numeric(df['edad'], errors='coerce')

# 3. Rellena nulos de edad con la mediana
df['edad'] = df['edad'].fillna(df['edad'].median())

# 4. Rellena nulos de salario con la media
df['salario'] = df['salario'].fillna(df['salario'].mean())

# 5. Normaliza ciudad a mayúsculas consistentes
df['ciudad'] = df['ciudad'].str.title().fillna('Desconocido

# 6. Elimina filas donde nombre es nulo
df = df.dropna(subset=['nombre'])

print("\n≡ DataFrame limpio ≡")
print(df.reset_index(drop=True))
```

Pista: `pd.to_numeric(col, errors="coerce")` convierte errores a NaN.
`str.title()` capitaliza correctamente.



MÓDULO COMPLETADO

Pandas te permite explorar, filtrar, agrupar y limpiar datos con pocas líneas de código. En el siguiente módulo aprenderás a *visualizar* esos datos — porque un gráfico revela patrones que no verías nunca en una tabla.

// LIBRO I — FUNDAMENTOS · MÓDULO 04 · 30 MIN · GRÁFICOS INTERACTIVOS

Matplotlib.

Visualizar datos revela lo que los números ocultan — y es esencial antes de entrenar cualquier modelo.

Antes de construir un modelo, siempre visualiza los datos. Un gráfico te puede decir en segundos si hay outliers, si hay una relación lineal o cuadrática, si los datos están desbalanceados, o si simplemente hay un error en el dataset. Sin visualización, entrenas a ciegas.

Matplotlib es la librería de visualización más usada en Python. Todos los gráficos que ves en papers de ML, tutoriales de data science y notebooks de Jupyter los genera (directa o indirectamente) Matplotlib. En este módulo aprenderás los cuatro tipos de gráfico que más usarás en ML — y los verás renderizados en tiempo real en el navegador.



PLT.SHOW() GENERA LA IMAGEN AQUÍ

En este libro, cuando escribes `plt.show()` el gráfico aparece directamente debajo del editor. No se abre ninguna ventana externa. Puedes modificar el código y volver a ejecutar para ver los cambios al instante.

Gráfico de líneas

Ideal para datos temporales: evolución de ventas, pérdida durante el entrenamiento, temperatura a lo largo del tiempo.

EJERCICIO 1

· fácil

Evolución de ventas mensuales

Dibuja la evolución de ventas de dos años para comparar la tendencia.

PYTHON

```
import matplotlib.pyplot as plt
import numpy as np

meses = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun', 'Jul', 'Ago', 'S
ventas_2023 = [420, 390, 510, 480, 560, 620, 590, 640, 580,
ventas_2024 = [450, 420, 540, 510, 600, 670, 640, 700, 650,

x = np.arange(len(meses))

plt.figure(figsize=(10, 5))
plt.plot(x, ventas_2023, marker='o', label='2023', color='#
plt.plot(x, ventas_2024, marker='s', label='2024', color='#

plt.xticks(x, meses)
plt.xlabel('Mes')
plt.ylabel('Ventas (miles FCFA)')
plt.title('Evolución de ventas – comparativa anual')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Pista: `plt.plot(x, y, marker="o", label="nombre")` dibuja la línea.
`plt.legend()` muestra la leyenda.

Gráfico de dispersión (scatter)

El más importante en ML para visualizar la **relación entre dos variables**. Antes de hacer una regresión, siempre haz un scatter plot para ver si la relación tiene sentido.

EJERCICIO 2

· fácil

Relación tamaño–precio de inmuebles

Visualiza la relación entre superficie y precio de inmuebles, diferenciando por zona.

PYTHON

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)
n = 80
metros = np.random.randint(40, 250, n)
# Precio base según metros + ruido
precio = metros * 350_000 + np.random.normal(0, 5_000_000, n)

# Clasificar por zona
zona = np.where(metros > 130, 'Zona norte', 'Zona sur')

plt.figure(figsize=(9, 6))
for z, color in [('Zona norte', '#1F4E5F'), ('Zona sur', '#E67E22')]:
    mask = zona == z
    plt.scatter(metros[mask], precio[mask] / 1_000_000,
                label=z, color=color, alpha=0.7, s=60)

plt.xlabel('Superficie (m²)')
plt.ylabel('Precio (millones FCFA)')
plt.title('Precio de inmuebles según superficie – Malabo')
plt.legend()
plt.grid(True, alpha=0.25)
plt.tight_layout()
plt.show()
```

Pista: Usa alpha para transparencia cuando los puntos se superponen. s controla el tamaño de los puntos.

Histograma

Muestra la **distribución** de una variable — cuántos valores caen en cada rango. Esencial para detectar sesgo, outliers y si los datos siguen una distribución normal.

EJERCICIO 3

· fácil

Distribución de salarios

Visualiza la distribución de salarios y añade líneas de referencia para media y mediana.

PYTHON

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(7)
# Salarios en FCFA con distribución asimétrica (realista)
salarios_bajos = np.random.normal(650_000, 80_000, 300)
salarios_medios = np.random.normal(1_100_000, 150_000, 150)
salarios_altos = np.random.normal(2_500_000, 400_000, 50)
todos = np.concatenate([salarios_bajos, salarios_medios, salarios_altos])

plt.figure(figsize=(10, 5))
plt.hist(todos / 1000, bins=40, color='#1F4E5F', alpha=0.75)

# Líneas de referencia
plt.axvline(np.median(todos)/1000, color='#D9742A', linestyle='solid')
plt.axvline(np.mean(todos)/1000, color='#C99419', linestyle='dashed')

plt.xlabel('Salario (miles FCFA)')
plt.ylabel('Frecuencia')
plt.title('Distribución de salarios')
plt.legend()
plt.tight_layout()
plt.show()
```

Pista: bins controla el número de barras. axvline dibuja una línea vertical en un valor X.

Subplots – múltiples gráficos en uno

EJERCICIO 4

· intermedio

Panel de análisis exploratorio

Crea un panel de 4 gráficos para el análisis exploratorio completo de un dataset de empleados.

PYTHON

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)
n = 150
edad = np.random.randint(18, 65, n)
salario = edad * 20_000 + np.random.normal(0, 100_000, n)
años_exp = np.maximum(0, edad - 22 + np.random.randint(-3, 3, n))
educacion = np.random.choice(['Secundaria', 'Grado', 'Máster', 'Doctorado'],
                              p=[0.30, 0.45, 0.20, 0.05])

fig, axes = plt.subplots(2, 2, figsize=(12, 9))
fig.suptitle('Análisis exploratorio – Dataset de empleados')

# 1. Distribución de edades
axes[0,0].hist(edad, bins=20, color='#1F4E5F', alpha=0.75,
               label='Distribución de edades'); axes[0,0].set_title('Distribución de edades'); axes[0,0].set_xlabel('Edad'); axes[0,0].set_ylabel('Frecuencia')

# 2. Salario vs edad
axes[0,1].scatter(edad, salario/1_000_000, alpha=0.5, color='blue',
                  label='Salario vs Edad'); axes[0,1].set_title('Salario vs Edad'); axes[0,1].set_xlabel('Edad'); axes[0,1].set_ylabel('Salario (Miles de $)')

# 3. Años de experiencia vs salario
axes[1,0].scatter(años_exp, salario/1_000_000, alpha=0.5, color='red',
                  label='Experiencia vs Salario'); axes[1,0].set_title('Experiencia vs Salario'); axes[1,0].set_xlabel('Años de experiencia'); axes[1,0].set_ylabel('Salario (Miles de $)')

# 4. Distribución por nivel educativo
conteo = {e: (educacion == e).sum() for e in ['Secundaria', 'Grado', 'Máster', 'Doctorado']}
axes[1,1].bar(conteo.keys(), conteo.values(), color=['#1F4E5F', '#E67E22', '#F1C40F', '#2E86C1'],
               label='Nivel educativo'); axes[1,1].set_title('Nivel educativo'); axes[1,1].set_xlabel('Nivel educativo'); axes[1,1].set_ylabel('Cantidad')

plt.tight_layout()
plt.show()
```

Pista: `fig, axes = plt.subplots(filas, columnas)` crea una cuadrícula de gráficos. Accede a cada uno con `axes[fila, columna]`.

◆ QUIZ RÁPIDO

¿Qué tipo de gráfico usarías para ver si existe una relación lineal entre dos variables numéricas?

A Histograma

B Gráfico de barras

C Gráfico de dispersión (scatter) ✓

D Gráfico de líneas

Respuesta — C. El scatter plot muestra cada par (x, y) como un punto. Si los puntos siguen una tendencia lineal, hay correlación. Es el gráfico de referencia para decidir si una regresión lineal tiene sentido.

◆ QUIZ RÁPIDO

Al visualizar datos antes de entrenar un modelo, ¿cuál es el objetivo principal?

A Hacer que los datos sean más grandes

B Detectar patrones, outliers y problemas antes de que el modelo los encuentre ✓

C Reducir el tiempo de entrenamiento del modelo

D Convertir los datos de CSV a formato NumPy

Respuesta — B. La visualización exploratoria es una fase de diagnóstico. Te dice si los datos tienen errores, relaciones inusuales, outliers o sesgos antes de invertir tiempo en entrenar un modelo con datos de mala calidad.

✓ MÓDULO COMPLETADO

Ahora puedes visualizar datos reales con cuatro tipos de gráfico esenciales en ML. En el siguiente módulo darás el salto más importante: construir tu primer modelo predictivo con regresión lineal.

// LIBRO I — FUNDAMENTOS · MÓDULO 05 · 40 MIN · PRIMER MODELO REAL

Regresión *lineal*.

Tu primer modelo predictivo: encontrar la línea que mejor describe la relación entre variables.

La **regresión lineal** es el algoritmo de ML más antiguo y más importante que aprenderás. No porque sea el más potente — hay muchos más precisos — sino porque sienta las bases conceptuales de todo lo que viene después: pérdida, optimización, entrenamiento, predicción.

La idea es simple: dada una nube de puntos, encontrar la línea recta que mejor los representa. Esa línea te permite predecir el valor de y para cualquier x nuevo que nunca viste.

La ecuación de la línea

Una línea recta se define por dos parámetros: **pendiente** (m) e **intercepto** (b):

PYTHON

```
# y = m * x + b
# Ejemplo: precio de casa = 350000 * metros + 5000000
# m = 350000 (precio por m² adicional)
# b = 5000000 (precio base sin metros)

metros = 80
precio = 350_000 * metros + 5_000_000
print(f"Casa de {metros}m²: {precio:,} FCFA")
```

El trabajo del algoritmo de regresión lineal es *encontrar* los valores óptimos de m y b que minimicen el error entre las predicciones y los valores reales.

El error: función de coste

¿Cómo mide el modelo qué tan buena es su línea? Con el **Error Cuadrático Medio (MSE)**: calcula la diferencia entre cada predicción y el valor real, la eleva al cuadrado (para que los errores negativos no cancelen a los positivos) y calcula la media.

PYTHON

```
import numpy as np

y_real = np.array([200, 350, 280, 410, 320])
y_pred = np.array([190, 360, 270, 420, 310])

errores = y_real - y_pred
mse = np.mean(errores ** 2)
rmse = np.sqrt(mse) # mismas unidades que y

print("Errores:", errores)
print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}") # → 11.40
```

Regresión lineal con scikit-learn

En la práctica, scikit-learn hace todo el trabajo de optimización por ti. El flujo siempre es el mismo: **crear** → **fit** → **predict**.

EJERCICIO 1

· fácil

Predecir precios de casas

Construye un modelo de regresión lineal para predecir precios de casas según los metros cuadrados. Evalúalo con R^2 y RMSE, y visualiza la línea de predicción.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Datos: m² → precio en millones FCFA
np.random.seed(42)
metros = np.random.randint(40, 300, 60)
precio = metros * 3.5 + np.random.normal(0, 20, 60) + 20 #

X = metros.reshape(-1, 1) # sklearn espera (n_muestras, n_
y = precio

# Separar en entrenamiento (80%) y prueba (20%)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, t

# Crear y entrenar el modelo
modelo = LinearRegression()
modelo.fit(X_train, y_train)

# Evaluar
y_pred = modelo.predict(X_test)
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"Pendiente (precio/m²): {modelo.coef_[0]:.2f} M FCFA")
print(f"Intercepto: {modelo.intercept_:.2f} M FCFA")
print(f"R² (precisión): {r2:.3f}")
print(f"RMSE (error típico): {rmse:.2f} M FCFA")

# Visualizar
plt.figure(figsize=(9, 5))
plt.scatter(X_test, y_test, color='#1F4E5F', alpha=0.7, label='Datos')
plt.plot(X_test, y_pred, color='#D9742A', linewidth=2.5, label='Regresión lineal')
plt.xlabel('Superficie (m²)'); plt.ylabel('Precio (M FCFA)')
plt.title(f'Regresión lineal - R²={r2:.3f}')
```

```
plt.legend(); plt.grid(True, alpha=0.3); plt.tight_layout()  
plt.show()
```

Pista: `reshape(-1, 1)` convierte un vector 1D a columna 2D, que sklearn requiere para X. `r2_score` de 1.0 sería perfecto; por encima de 0.85 es muy bueno.

Regresión múltiple — varias características

Pocas veces una sola variable predice bien el resultado. La regresión **múltiple** usa varias características a la vez:

EJERCICIO 2

· intermedio

Regresión múltiple: precio con metros, habitaciones y antigüedad

Usa tres características para predecir el precio de inmuebles. Compara los coeficientes para entender qué variable influye más.

PYTHON

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler

np.random.seed(0)
n = 200

# Generar dataset sintético de inmuebles
metros = np.random.randint(40, 300, n)
habitaciones = np.random.randint(1, 6, n)
antigüedad = np.random.randint(0, 40, n)

# Precio = función de las 3 variables + ruido
precio = (metros * 3.5 +
          habitaciones * 15 -
          antigüedad * 2 +
          np.random.normal(0, 15, n) + 30)

df = pd.DataFrame({'metros': metros, 'habitaciones': habitaciones,
                  'antigüedad': antigüedad, 'precio': precio})

X = df[['metros', 'habitaciones', 'antigüedad']]
y = df['precio']

# Escalar features (importante en regresión múltiple)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
                                                    test_size=0.2,
                                                    random_state=0)

modelo = LinearRegression()
modelo.fit(X_train, y_train)

r2 = r2_score(y_test, modelo.predict(X_test))
print(f"R²: {r2:.3f}")
print("\nCoeficientes (importancia de cada feature):")
```

```
for feature, coef in zip(['metros', 'habitaciones', 'antiguedad']):
    print(f" {feature:15s}: {coef:+.2f}")

# Predecir una casa específica
casa_nueva = scaler.transform([[120, 3, 10]]) # 120m², 3 hab, 10 años
pred = modelo.predict(casa_nueva)[0]
print(f"\nPredicción 120m², 3 hab, 10 años: {pred:.1f} M FC")
```

Pista: StandardScaler normaliza las features para que todas tengan media 0 y desviación estándar 1. Importante cuando las features tienen escalas muy diferentes.

◆ QUIZ RÁPIDO

Un modelo de regresión lineal tiene $R^2=0.92$. ¿Qué significa?

- A El modelo tiene un 92% de probabilidad de estar bien entrenado
- B El modelo explica el 92% de la varianza de la variable objetivo ✓**
- C El error promedio del modelo es del 8%
- D El modelo predice correctamente 92 de cada 100 valores

Respuesta — B. R^2 (coeficiente de determinación) mide qué proporción de la variabilidad en y es explicada por el modelo. $R^2=0.92$ significa que el modelo captura el 92% de los patrones en los datos. 1.0 sería perfecto.

◆ QUIZ RÁPIDO

¿Por qué usamos `train_test_split` antes de evaluar el modelo?

- A Para que el entrenamiento sea más rápido
- B Para evaluar el modelo con datos que nunca vio durante el entrenamiento ✓**
- C Porque sklearn lo requiere obligatoriamente
- D Para reducir el uso de memoria RAM

Respuesta — B. Evaluar con los mismos datos de entrenamiento daría una falsa sensación de precisión. El modelo memoriza los datos de train. Solo los datos de test (que nunca vio) revelan si realmente generaliza.

✓ MÓDULO COMPLETADO

Has entrenado y evaluado tu primer modelo de regresión lineal real. Entiendes los conceptos clave: coste, entrenamiento, evaluación con R^2 y RMSE, y la importancia del train/test split. En el siguiente módulo pasaremos a la *clasificación* — cuando la respuesta no es un número continuo sino una categoría.

// LIBRO I — FUNDAMENTOS · MÓDULO 06 · 35 MIN · CLASIFICACIÓN VISUAL

Clasificación.

Cuando la respuesta no es un número sino una categoría — y el algoritmo que aprende por vecindad.

En el módulo anterior predijimos un *número continuo* (el precio de una casa). Pero muchos problemas reales requieren predecir una **categoría**: ¿este email es spam o no? ¿este cliente va a cancelar su suscripción? ¿este tumor es maligno o benigno? Eso es **clasificación**.

El primer clasificador que aprenderás es **K-Nearest Neighbors (KNN)** — literalmente "K vecinos más cercanos". La idea es completamente intuitiva: para clasificar un punto nuevo, mira cuáles son sus K puntos más cercanos en el dataset de entrenamiento, y asígnale la categoría más frecuente entre esos vecinos.

Cómo funciona KNN

Imagina que tienes pacientes clasificados como "sano" o "enfermo" según dos análisis de sangre. Para clasificar a un paciente nuevo:

1. Calcula la distancia entre el nuevo paciente y todos los del dataset
2. Selecciona los K más cercanos
3. La categoría que más se repite entre los K vecinos es la predicción

PYTHON

```
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

# Dataset: [glucosa, presión] → [0=sano, 1=en riesgo]
X_train = np.array([
    [85, 70], [90, 75], [95, 80], # sanos
    [140, 95], [160, 105], [150, 100], # en riesgo
])
y_train = np.array([0, 0, 0, 1, 1, 1])

# Crear y entrenar el modelo con K=3
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# Clasificar un nuevo paciente
nuevo = np.array([[120, 88]])
clase = knn.predict(nuevo)[0]
proba = knn.predict_proba(nuevo)[0]

etiquetas = {0: 'Sano', 1: 'En riesgo'}
print(f"Paciente con glucosa=120, presión=88:")
print(f"  Clasificado como: {etiquetas[clase]}")
print(f"  Probabilidad sano: {proba[0]:.0%}")
print(f"  Probabilidad en riesgo: {proba[1]:.0%}")
```

Eligiendo K – el hiperparámetro clave

K es un **hiperparámetro**: un valor que tú decides antes del entrenamiento. K pequeño → el modelo es sensible al ruido (overfitting). K grande → el modelo es demasiado general (underfitting). La forma correcta de elegir K es probando diferentes valores y midiendo el accuracy en los datos de test.

EJERCICIO 1

· fácil

Encontrar el K óptimo

Prueba diferentes valores de K en el dataset Iris y visualiza cómo cambia el accuracy. Identifica el K óptimo.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Dataset Iris: clasificar 3 especies de flores por sus medidas
iris = load_iris()
X, y = iris.data, iris.target

# Escalar features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.25, random_state=42
)

# Probar K del 1 al 25
k_valores = range(1, 26)
accuracy_train = []
accuracy_test = []

for k in k_valores:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    accuracy_train.append(knn.score(X_train, y_train))
    accuracy_test.append(knn.score(X_test, y_test))

# Mejor K
mejor_k = k_valores[np.argmax(accuracy_test)]
mejor_acc = max(accuracy_test)
print(f"Mejor K: {mejor_k} → Accuracy test: {mejor_acc:.3f}")

# Gráfico
plt.figure(figsize=(10, 5))
plt.plot(k_valores, accuracy_train, 'o-', color='#1F4E5F',
plt.plot(k_valores, accuracy_test, 's-', color='#D9742A',
```

```
plt.axvline(mejor_k, color='#C99419', linestyle='--', label=mejor_k)
plt.xlabel('Valor de K'); plt.ylabel('Accuracy')
plt.title('Accuracy según K – Dataset Iris')
plt.legend(); plt.grid(True, alpha=0.3); plt.tight_layout()
plt.show()
```

Pista: `knn.score(X, y)` devuelve el accuracy directamente. `np.argmax()` encuentra el índice del valor máximo.

EJERCICIO 2

· intermedio

Clasificador de riesgo crediticio

Clasifica solicitudes de crédito como aprobadas o rechazadas usando KNN. Evalúa el modelo con métricas específicas de clasificación.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

np.random.seed(42)
n = 300

# Features: [ingresos_mensual (k FCFA), deuda_actual (k FCFA)]
ingresos = np.random.normal(900, 250, n).clip(200, 3000)
deuda = np.random.normal(400, 200, n).clip(0, 2000)
años_emp = np.random.randint(0, 20, n)

# Etiqueta: aprobado (1) si ingresos altos, deuda baja y ex
ratio_deuda = deuda / ingresos
aprobado = ((ratio_deuda < 0.45) & (ingresos > 600) & (años_emp > 5))
# Añadir algo de ruido
flip = np.random.rand(n) < 0.08
aprobado[flip] = 1 - aprobado[flip]

X = np.column_stack([ingresos, deuda, años_emp])
y = aprobado

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

print(f"Accuracy: {knn.score(X_test, y_test):.3f}")
print()
print("Reporte de clasificación:")
print(classification_report(y_test, y_pred, target_names=['Aprobado', 'No Aprobado']))

# Visualizar distribución de predicciones
```

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
# Scatter: ingresos vs deuda, coloreado por predicción
axes[0].scatter(ingresos[:len(y_test)], deuda[:len(y_test)]
                c=y_pred, cmap='coolwarm', alpha=0.6, s=30)
axes[0].set_xlabel('Ingresos (k FCFA)'); axes[0].set_ylabel('Deuda (k FCFA)')
axes[0].set_title('Predicciones del clasificador')
# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
axes[1].imshow(cm, cmap='Blues')
for i in range(2):
    for j in range(2):
        axes[1].text(j, i, cm[i,j], ha='center', va='center')
axes[1].set_xticks([0,1]); axes[1].set_yticks([0,1])
axes[1].set_xticklabels(['Rechazado', 'Aprobado']); axes[1].set_yticklabels(['Rechazado', 'Aprobado'])
axes[1].set_xlabel('Predicción'); axes[1].set_ylabel('Real')
axes[1].set_title('Matriz de confusión')
plt.tight_layout(); plt.show()
```

Pista: classification_report muestra precisión, recall y F1 para cada clase. La confusion matrix muestra los cuatro tipos de error posibles.

◆ QUIZ RÁPIDO

En KNN, ¿qué ocurre cuando usas $K=1$?

- A El modelo no puede hacer predicciones
- B El modelo usa solo el vecino más cercano → puede memorizar el ruido del dataset ✓**
- C $K=1$ es siempre el mejor valor porque es el más preciso
- D El modelo necesita normalizar los datos antes de funcionar

Respuesta — B. Con $K=1$, el modelo asigna a cada punto la clase de su vecino más próximo. En entrenamiento tendrá accuracy perfecta (100%), pero fallará con datos nuevos porque habrá sobreajustado al ruido. Esto se llama overfitting.

◆ QUIZ RÁPIDO

¿Por qué es importante normalizar las features antes de aplicar KNN?

A KNN no funciona sin normalización, dará error

B Porque KNN basa sus predicciones en distancias, y features con rangos grandes dominarían ✓

C Para reducir el tiempo de predicción

D Porque scikit-learn lo requiere en su API

Respuesta — B. KNN calcula distancias euclidianas. Si una feature va de 0-10 y otra de 0-1.000.000, la segunda domina la distancia y la primera se ignora. Normalizar pone todas las features en el mismo rango de influencia.

✓ MÓDULO COMPLETADO

KNN es tu primer clasificador. En el siguiente módulo aprenderás los *árboles de decisión*— un algoritmo que aprende reglas explícitas que puedes leer y entender, y que lleva a uno de los algoritmos más potentes del ML clásico: Random Forest.

// LIBRO I — FUNDAMENTOS · MÓDULO 07

· 40 MIN · EL ALGORITMO MÁS USADO EN PRODUCCIÓN

Árboles y *Random Forest.*

El algoritmo que aprende reglas legibles — y cómo cien árboles juntos superan a cualquiera por separado.

Los **árboles de decisión** son los modelos más intuitivos del ML. Imitan exactamente cómo tomamos decisiones en la vida real: una serie de preguntas binarias que llevan a una conclusión. ¿Tiene fiebre? Sí → ¿Tos seca? Sí → *Probablemente viral.*

La diferencia con KNN o la regresión es que un árbol *aprende las reglas explícitamente*. Puedes imprimirlo, leerlo y explicar cada predicción. En sectores como la banca, la salud o el sector legal, esto es a menudo más valioso que la precisión bruta.

Cómo aprende un árbol

En cada nodo del árbol, el algoritmo elige la pregunta (la división del espacio) que mejor separa las clases. El criterio más común es la **impureza de Gini**: mide cuán mezcladas están las clases en cada nodo. A mayor pureza, mejor la división.

PYTHON

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Árbol sin límite → memorizará todo (overfitting)
arbol_profundo = DecisionTreeClassifier(random_state=42)
arbol_profundo.fit(X_train, y_train)

# Árbol limitado en profundidad → mejor generalización
arbol_podado = DecisionTreeClassifier(max_depth=3, random_state=42)
arbol_podado.fit(X_train, y_train)

print("Sin límite – Train:", arbol_profundo.score(X_train, y_train))
print("Sin límite – Test: ", arbol_profundo.score(X_test, y_test))
print()
print("Podado (depth=3) – Train:", arbol_podado.score(X_train, y_train))
print("Podado (depth=3) – Test: ", arbol_podado.score(X_test, y_test))
```

 OVERFITTING EN ÁRBOLES

Un árbol sin límite de profundidad memoriza el dataset de entrenamiento — incluyendo el ruido. En entrenamiento tendrá 100% de accuracy, pero fallará con datos nuevos. Limitar la profundidad con `max_depth` es la forma más directa de combatir el overfitting.

Importancia de features

Una de las grandes ventajas de los árboles: te dicen qué variables son más importantes para la predicción. Esto es crucial para entender el problema y para limpiar features irrelevantes.

EJERCICIO 1

· fácil

Importancia de variables en predicción de diabetes

Visualiza qué variables son más importantes para predecir la progresión de diabetes usando un árbol de decisión.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import Binarizer

# Usamos el dataset de diabetes (regresión → lo convertimos
datos = load_diabetes()
X = datos.data
# Binarizamos: 1 si progresión alta, 0 si baja
umbral = np.median(datos.target)
y = (datos.target > umbral).astype(int)

feature_names = ['edad', 'sexo', 'IMC', 'PA', 'S1', 'S2', 'S3', 'S

X_train, X_test, y_train, y_test = train_test_split(X, y, t

arbol = DecisionTreeClassifier(max_depth=4, random_state=42)
arbol.fit(X_train, y_train)

print(f"Accuracy: {arbol.score(X_test, y_test):.3f}")
print()

# Importancia de features
importancias = arbol.feature_importances_
orden = np.argsort(importancias)[::-1]

print("Importancia de cada variable:")
for i in orden:
    bar = '█' * int(importancias[i] * 40)
    print(f" {feature_names[i]:5s}: {importancias[i]:.3f}")

# Gráfico
plt.figure(figsize=(9, 5))
plt.barh([feature_names[i] for i in orden[::-1]],
         [importancias[i] for i in orden[::-1]],
         color='#1F4E5F', alpha=0.8)
plt.xlabel('Importancia')
```

```
plt.title('Importancia de features – Árbol de decisión')  
plt.tight_layout()  
plt.show()
```

Pista: `feature_importances_` es un array con la importancia relativa de cada feature. `np.argsort()[::-1]` ordena de mayor a menor.

Random Forest – la sabiduría de la multitud

Un árbol individual es inestable: pequeñas variaciones en los datos pueden producir árboles muy distintos. La solución: entrenar *cientos de árboles* en versiones ligeramente diferentes del dataset, y combinar sus predicciones. A esto se llama **ensemble**.

Random Forest hace exactamente eso: cada árbol se entrena con una muestra aleatoria del dataset (bootstrapping) y en cada nodo solo considera un subconjunto aleatorio de features. El resultado es mucho más robusto y preciso que cualquier árbol individual.

EJERCICIO 2

· intermedio

Random Forest vs árbol individual

Compara el accuracy de un árbol individual vs Random Forest con validación cruzada. Observa cómo el ensemble mejora tanto la precisión como la estabilidad.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
from sklearn.datasets import make_classification

# Dataset sintético de clasificación binaria
X, y = make_classification(
    n_samples=500, n_features=10, n_informative=6,
    n_redundant=2, random_state=42
)

modelos = {
    'Árbol (depth=3)': DecisionTreeClassifier(max_depth=3,
    'Árbol (sin límite)': DecisionTreeClassifier(random_state=42),
    'Random Forest (50 árboles)': RandomForestClassifier(n_estimators=50),
    'Random Forest (200 árboles)': RandomForestClassifier(n_estimators=200)
}

resultados = {}
for nombre, modelo in modelos.items():
    scores = cross_val_score(modelo, X, y, cv=5, scoring='accuracy')
    resultados[nombre] = scores
    print(f"{nombre:35s} {scores.mean():.3f} ± {scores.std():.3f}")

# Gráfico comparativo
plt.figure(figsize=(10, 5))
nombres = list(resultados.keys())
medias = [resultados[n].mean() for n in nombres]
errors = [resultados[n].std() for n in nombres]
colores = ['#C99419', '#A8331E', '#1F4E5F', '#3F6B3B']

bars = plt.barh(nombres, medias, xerr=errors, color=colores)
plt.xlabel('Accuracy (validación cruzada 5-fold)')
plt.title('Comparativa: árbol individual vs Random Forest')
plt.xlim(0.6, 1.0)
```

```
plt.tight_layout()
plt.show()
```

Pista: `cross_val_score` evalúa el modelo en 5 particiones distintas. La media $\pm$ desviación estándar es una medida más fiable que una sola evaluación.

◆ QUIZ RÁPIDO

¿Por qué Random Forest es más robusto que un árbol de decisión individual?

A Porque usa más memoria RAM y puede procesar más datos

B Porque combina las predicciones de muchos árboles entrenados en muestras y features aleatorias ✓

C Porque aplica normalización automática antes de entrenar

D Porque Random Forest no puede hacer overfitting nunca

Respuesta — B. La diversidad forzada (muestras aleatorias + features aleatorias por nodo) hace que cada árbol cometa errores distintos. Al promediar, los errores se cancelan parcialmente y el resultado es más estable y preciso.

◆ QUIZ RÁPIDO

Un árbol de decisión sin límite de profundidad obtiene 100% de accuracy en training y 68% en test. ¿Qué ocurre?

- A El modelo es perfecto y necesita más datos de test
- B Hay overfitting: el árbol memorizó el training en lugar de aprender patrones generalizables ✓**
- C El dataset tiene muy pocos ejemplos de test
- D El problema es demasiado difícil para un árbol de decisión

Respuesta — B. 100% en train y 68% en test es el síntoma clásico de overfitting. El árbol se adaptó perfectamente al ruido del training set y no generaliza. Solución: limitar `max_depth`, `min_samples_leaf`, o usar Random Forest.

✓ MÓDULO COMPLETADO

Entiendes los árboles de decisión, el problema del overfitting y por qué los ensembles como Random Forest son superiores. En el último módulo del Libro 1 aprenderás a medir correctamente si tu modelo realmente funciona — el paso que separa los proyectos de juguete de los sistemas en producción.

// LIBRO I — FUNDAMENTOS · MÓDULO 08 · 35 MIN · CIERRE DEL LIBRO 1

Evaluación *de modelos.*

Un modelo que no sabe que se equivoca es peor que ningún modelo. Aprende a medir correctamente.

Has aprendido a construir modelos. Ahora aprenderás la parte más importante: **saber si realmente funcionan**. Un modelo que parece preciso en tus pruebas pero falla en producción puede ser peor que no tener modelo — porque tomas decisiones basadas en predicciones incorrectas.

El problema del data leakage

El error más grave en ML. Ocurre cuando información del futuro (o del test set) contamina el entrenamiento. El modelo aprende a "hacer trampa" y parece tener una precisión irreal.

PYTHON

```
# MAL EJEMPLO — data leakage
from sklearn.preprocessing import StandardScaler
import numpy as np

X = np.random.randn(100, 5)
y = np.random.randint(0, 2, 100)

# ❌ ERROR: escalar ANTES de dividir mezcla info del test en el train
scaler = StandardScaler()
X_escalado = scaler.fit_transform(X) # usa estadísticas de TODO el dataset

from sklearn.model_selection import train_test_split
X_train, X_test = train_test_split(X_escalado)

# ✅ CORRECTO: dividir primero, escalar después
X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_raw) # aprende de train
X_test = scaler.transform(X_test_raw)      # aplica sin reaprender
```

Métricas de clasificación

Accuracy (porcentaje de aciertos) es la métrica más simple, pero puede ser engañosa cuando las clases están desbalanceadas. Si el 95% de tus emails son legítimos, un modelo que nunca detecta spam tiene 95% de accuracy — y es completamente inútil.

PYTHON

```

# Las cuatro métricas esenciales de clasificación

# Precision: de todos los que dije que son positivos, ¿cuántos lo son?
# → "¿Cuándo digo que es fraude, estoy en lo correcto?"

# Recall (sensibilidad): de todos los positivos reales, ¿cuántos encontre?
# → "¿Cuánto fraude real detecté?"

# F1-Score: media armónica de precision y recall
# → Útil cuando quieres equilibrar ambas métricas

# La elección depende del problema:
# - Diagnóstico médico: maximiza Recall (no puedes perder enfermos)
# - Filtro de spam: maximiza Precision (no quieres perder emails importantes)

```

#	métrica	qué_mide	cuando_usarla
0	Accuracy	% predicciones correctas	Clases balanceadas
1	Precision	De los "positivos" predichos, % reales	Coste alto de falsos positivos
2	Recall	De los positivos reales, % detectados	Coste alto de falsos negativos
3	F1-Score	Balance entre precision y recall	Clases desbalanceadas
4	ROC-AUC	Capacidad discriminativa general	Comparar modelos diferentes

Guía rápida para elegir la métrica correcta según el problema

EJERCICIO 1

· fácil

Evaluación completa de un clasificador de fraude

Evalúa un detector de fraude bancario con todas las métricas relevantes: matriz de confusión, precision/recall por clase y curva ROC.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, roc_curve)

np.random.seed(42)
n = 1000

# Dataset desbalanceado: 5% de transacciones son fraude
X = np.random.randn(n, 6)
# Clase 1 (fraude) → puntos en extremos del espacio de features
es_fraude = (np.abs(X[:, 0]) > 1.5) & (np.abs(X[:, 2]) > 1.5)
y = es_fraude.astype(int)
print(f"Distribución: {(y==0).sum()} legítimas / {(y==1).sum()} fraudes")

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

rf = RandomForestClassifier(n_estimators=100, class_weight='balanced')
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
y_proba = rf.predict_proba(X_test)[:, 1]

print(f"\nAccuracy: {(y_pred == y_test).mean():.3f}")
print(f"ROC-AUC: {roc_auc_score(y_test, y_proba):.3f}")
print()
print(classification_report(y_test, y_pred, target_names=['legítima', 'fraude']))

# Gráficos
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
im = axes[0].imshow(cm, cmap='Blues')
for i in range(2):
    for j in range(2):
        color = 'white' if cm[i,j] > cm.max()/2 else 'black'
        axes[0].text(j, i, f'{cm[i,j]}', ha='center', va='center', color=color)
```

```
axes[0].set_xticks([0,1]); axes[0].set_yticks([0,1])
axes[0].set_xticklabels(['Legítima','Fraude']); axes[0].set
axes[0].set_xlabel('Predicción'); axes[0].set_ylabel('Real'
axes[0].set_title('Matriz de confusión')

# Curva ROC
fpr, tpr, _ = roc_curve(y_test, y_proba)
auc = roc_auc_score(y_test, y_proba)
axes[1].plot(fpr, tpr, color='#1F4E5F', linewidth=2.5, label=
axes[1].plot([0,1],[0,1], 'k--', alpha=0.4, label='Azar')
axes[1].set_xlabel('Tasa de Falsos Positivos'); axes[1].set
axes[1].set_title('Curva ROC'); axes[1].legend(); axes[1].g

plt.tight_layout(); plt.show()
```

Pista: `class_weight="balanced"` hace que el modelo dé más peso a la clase minoritaria (fraude). Crucial en datasets desbalanceados.

Validación cruzada — una evaluación más honesta

Con un solo train/test split, el resultado depende de qué datos cayeron en cada parte. La **validación cruzada k-fold** reparte los datos en k partes, entrena k veces (cada vez dejando una parte distinta como test) y promedia. Es mucho más fiable.

EJERCICIO 2

· intermedio

Comparar tres modelos con validación cruzada

Compara tres modelos con validación cruzada de 10 particiones y múltiples métricas. ¿Cuál es mejor para diagnóstico médico?

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import cross_validate
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler

# Dataset de cáncer de mama – clasificación binaria médica
datos = load_breast_cancer()
X, y = datos.data, datos.target

scaler = StandardScaler()
X_norm = scaler.fit_transform(X)

modelos = {
    'Árbol (depth=4)': DecisionTreeClassifier(max_depth=4,
    'KNN (K=7)': KNeighborsClassifier(n_neighbors=7),
    'Random Forest': RandomForestClassifier(n_estimators=
}

metricas = ['accuracy', 'precision', 'recall', 'f1']
resultados = {}

for nombre, modelo in modelos.items():
    cv = cross_validate(modelo, X_norm, y, cv=10, scoring=m
    resultados[nombre] = {m: cv[f'test_{m}']} for m in metri
    print(f"{nombre}:")
    for m in metricas:
        scores = resultados[nombre][m]
        print(f" {m:10s}: {scores.mean():.3f} ± {scores.st
    print()

# Gráfico comparativo
fig, axes = plt.subplots(1, 4, figsize=(14, 5))
for i, m in enumerate(metricas):
    datos_plot = [[resultados[n][m]] for n in modelos]
    axes[i].boxplot([resultados[n][m] for n in modelos], la
```

```
axes[i].set_title(m.capitalize())
axes[i].set_ylim(0.8, 1.0)
axes[i].grid(True, alpha=0.3)
plt.suptitle('Comparativa modelos – Validación cruzada 10-f
plt.tight_layout(); plt.show()
```

Pista: cross_validate con múltiples métricas devuelve un dict con test_accuracy, test_precision, etc. Usa cv=10 para 10 particiones.

◆ QUIZ RÁPIDO

Un modelo de detección de enfermedades tiene Accuracy=98% pero Recall=40%. ¿Es un buen modelo?

A

Sí, 98% de accuracy es excelente para cualquier problema médico

B

No — Recall=40% significa que el modelo falla al detectar el 60% de los enfermos reales ✓

C

Depende de cuántos datos tiene el training set

D

Sí, mientras la precision sea alta el recall no importa

Respuesta — B. En diagnóstico médico, el Recall (sensibilidad) es crítico: mide qué % de enfermos reales detecta el modelo. Recall=40% significa que el 60% de los enfermos pasarían como sanos — desastroso en medicina. La alta accuracy puede ser engañosa si hay muchos negativos (sanos).

◆ QUIZ RÁPIDO

¿Cuál es la principal ventaja de la validación cruzada sobre un solo train/test split?

- A Es más rápida de ejecutar
- B Usa más datos para entrenamiento siempre
- C Da una estimación más estable del rendimiento real, al promediar sobre múltiples particiones ✓**
- D Evita automáticamente el overfitting

Respuesta — C. Con un solo split, la evaluación depende de qué datos cayeron en test por azar. Con 10-fold cross-validation, evalúas sobre el dataset completo en 10 iteraciones. La media y desviación estándar reflejan mucho mejor el rendimiento real del modelo.



¡LIBRO 1 COMPLETADO!

Has dominado el stack fundamental del ML en Python:

- NumPy para manipulación de arrays y álgebra lineal
- Pandas para exploración y limpieza de datos
- Matplotlib para visualización
- Regresión lineal, KNN, árboles de decisión y Random Forest
- Evaluación correcta con métricas, confusión matrix y validación cruzada

El Libro 2 te lleva a técnicas más avanzadas: preprocesamiento, SVM, clustering, PCA y pipelines. El Libro 3 entra en redes neuronales profundas y los modelos que están cambiando el mundo.



Técnicas Avanzadas

ML con profundidad, proyectos reales y pipelines

- 01. Preprocesamiento de datos · 40 min
- 02. Regresión con regularización · 35 min
- 03. Máquinas de soporte vectorial · 40 min
- 04. Clustering sin etiquetas · 35 min
- 05. PCA y reducción de dimensiones · 35 min
- 06. Selección de características · 30 min
- 07. Pipelines y automatización · 40 min
- 08. Proyecto: sistema de recomendación · 50 min

// LIBRO II — TÉCNICAS AVANZADAS · MÓDULO 01

· 40 MIN · LA BASE DE TODO MODELO REAL

Preprocesamiento *de datos.*

*El 80% del trabajo real en ML no es el modelo
— es limpiar, transformar y preparar los
datos correctamente.*

En los ejercicios del Libro 1 usamos datasets de scikit-learn ya limpios y bien formateados. En el mundo real, los datos vienen con valores nulos, categorías como texto, escalas dispares y outliers que rompen los modelos. Este módulo te enseña a manejar todos esos casos.

Variables categóricas: encoding

Los modelos de ML solo trabajan con números. Cuando tienes una columna "ciudad" con valores como "Madrid", "Barcelona", "Valencia", necesitas convertirla. Hay tres estrategias principales, y elegir la incorrecta puede introducir sesgos artificiales.

#	técnica	qué_hace	cuándo_usarla
0	Label Encoding	Madrid=0, Barcelona=1, Valencia=2	Variables ordinales (Malo < Regular < Bueno)
1	One-Hot Encoding	Columna binaria por categoría	Variables nominales sin orden (ciudad, color)
2	Ordinal Encoding	Asigna enteros respetando orden definido	Variables con orden explícito (bajo/medio/alto)

Cómo convertir variables categóricas a números

EJERCICIO 1

· fácil

Encoding de variables categóricas

Compara las tres técnicas de encoding en un dataset de clientes. Observa por qué Label Encoding en variables nominales puede introducir relaciones falsas.

PYTHON

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Dataset de ejemplo: clientes de una tienda
datos = pd.DataFrame({
    'ciudad':    ['Madrid', 'Barcelona', 'Valencia', 'Madrid', 'Barcelona'],
    'nivel':     ['Alto', 'Bajo', 'Medio', 'Medio', 'Alto'],
    'genero':    ['M', 'F', 'F', 'M', 'F'],
    'compras':   [150, 80, 120, 200, 95]
})

print("Dataset original:")
print(datos)
print()

# 1. Label Encoding (solo para demostrar – MAL para 'ciudad')
le = LabelEncoder()
datos_le = datos.copy()
datos_le['ciudad_le'] = le.fit_transform(datos['ciudad'])
print("Label Encoding (ciudad):", dict(zip(le.classes_, le.transform(datos_le['ciudad_le']))))

# 2. One-Hot Encoding (correcto para 'ciudad' y 'genero')
datos_ohe = pd.get_dummies(datos, columns=['ciudad', 'genero'])
print()
print("One-Hot Encoding:")
print(datos_ohe)

# 3. Ordinal Encoding (correcto para 'nivel')
oe = OrdinalEncoder(categories=[['Bajo', 'Medio', 'Alto']])
datos_ord = datos.copy()
datos_ord['nivel_num'] = oe.fit_transform(datos[['nivel']])
print()
print("Ordinal Encoding (nivel):")
print(datos_ord[['nivel', 'nivel_num']])
```

Pista: `pd.get_dummies()` es la forma más rápida de aplicar One-Hot Encoding con pandas. `OrdinalEncoder` acepta el orden explícito

de categorías.

Outliers: detección y tratamiento

Un outlier es un valor atípico que se aleja significativamente del resto. Puede ser un error de medición (glucosa = 9999) o un caso real extremo. Los outliers afectan gravemente a la regresión lineal y a los modelos basados en distancias como KNN.

El método más robusto es el **rango intercuartílico (IQR)**: valores por debajo de $Q1 - 1.5 \cdot IQR$ o por encima de $Q3 + 1.5 \cdot IQR$ se consideran atípicos.

EJERCICIO 2

· fácil

Detección y manejo de outliers con IQR

Detecta outliers en un dataset de salarios con el método IQR y compara tres estrategias de tratamiento: eliminar, recortar e imputar con la mediana.

PYTHON

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)
n = 200

# Dataset de salarios con outliers artificiales
salarios_normales = np.random.normal(2800, 600, n)
# Añadir outliers: errores de entrada de datos
outliers = np.array([25000, 30000, -500, 35000, 22000])
salarios = np.concatenate([salarios_normales, outliers])

df = pd.DataFrame({'salario': salarios})

# Calcular límites IQR
Q1 = df['salario'].quantile(0.25)
Q3 = df['salario'].quantile(0.75)
IQR = Q3 - Q1
lim_inf = Q1 - 1.5 * IQR
lim_sup = Q3 + 1.5 * IQR

es_outlier = (df['salario'] < lim_inf) | (df['salario'] > lim_sup)
print(f"Q1={Q1:.0f}, Q3={Q3:.0f}, IQR={IQR:.0f}")
print(f"Límites: [{lim_inf:.0f}, {lim_sup:.0f}]")
print(f"Outliers detectados: {es_outlier.sum()} de {len(df)}")
print()

# Estrategias de tratamiento
df_clip = df.copy(); df_clip['salario'] = df['salario'].clip(lower=lim_inf, upper=lim_sup)
df_remove = df[~es_outlier].copy()
df_median = df.copy(); df_median.loc[es_outlier, 'salario'] = df['salario'].median()

print(f"Original - media: {df['salario'].mean():.0f}, mediana: {df['salario'].median():.0f}")
print(f"Eliminado - media: {df_remove['salario'].mean():.0f}, mediana: {df_remove['salario'].median():.0f}")
print(f"Recortado - media: {df_clip['salario'].mean():.0f}, mediana: {df_clip['salario'].median():.0f}")

# Visualización
fig, axes = plt.subplots(1, 3, figsize=(13, 4))
```

```
for ax, d, title in zip(axes,
                        [df['salario'], df_remove['salario'], df_clip['salario'],
                         ['Original (con outliers)', 'Outliers eliminados', 'Outliers eliminados']],
                        ['Original (con outliers)', 'Outliers eliminados', 'Outliers eliminados']):
    ax.boxplot(d, vert=True)
    ax.set_title(title)
    ax.set_ylabel('Salario (FCFA)')
plt.tight_layout()
plt.show()
```

Pista: `df.clip(lower, upper)` recorta los valores al rango dado. `df[~mask]` filtra filas donde la máscara es `False`.

Datos desbalanceados

Cuando una clase es mucho más frecuente que otra (por ejemplo, 98% transacciones legítimas vs 2% fraude), los modelos tienden a ignorar la clase minoritaria. Hay tres formas de combatirlo:

EJERCICIO 3

· intermedio

`class_weight` vs SMOTE en clasificación desbalanceada

Compara tres configuraciones de Random Forest en un dataset desbalanceado 95/5. Observa cómo `class_weight` mejora la detección de la clase minoritaria.

PYTHON

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_validate
from sklearn.metrics import f1_score, make_scorer
from sklearn.datasets import make_classification

np.random.seed(42)

# Dataset muy desbalanceado: 95% negativos, 5% positivos
X, y = make_classification(
    n_samples=1000, n_features=8, n_informative=5,
    weights=[0.95, 0.05], random_state=42
)
print(f"Distribución: {(y==0).sum()} negativos / {(y==1).sum()} positivos")
print()

f1_minority = make_scorer(f1_score, pos_label=1)

modelos = {
    'Sin ajuste': RandomForestClassifier(n_estimators=100),
    'class_weight=balanced': RandomForestClassifier(n_estimators=100, class_weight='balanced'),
    'class_weight manual': RandomForestClassifier(n_estimators=100, class_weight={0: 1, 1: 19})
}

resultados = {}
for nombre, modelo in modelos.items():
    cv = cross_validate(modelo, X, y, cv=5,
                        scoring={'accuracy': 'accuracy', 'f1': f1_minority})
    resultados[nombre] = cv
    print(f"{nombre}:")
    print(f"  Accuracy: {cv['test_accuracy'].mean():.3f} ± {cv['test_accuracy'].std():.3f}")
    print(f"  F1 clase minoritaria: {cv['test_f1'].mean():.3f} ± {cv['test_f1'].std():.3f}")
    print()

# Gráfico comparativo
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
nombres = list(resultados.keys())
for i, (metrica, titulo) in enumerate([('test_accuracy', 'Accuracy'), ('test_f1', 'F1')]):
    for nombre in nombres:
        resultados[nombre][metrica].plot(kind='box', ax=axes[i][0])

```

```
axes[i].boxplot([resultados[n][metrica] for n in nombre
                 labels=['Sin ajuste', 'Balanced', 'Manu
axes[i].set_title(titulo)
axes[i].grid(True, alpha=0.3)
plt.suptitle('Impacto de class_weight en dataset desbalance
plt.tight_layout()
plt.show()
```

Pista: `class_weight="balanced"` ajusta automáticamente los pesos inversamente proporcionales a la frecuencia de cada clase. El F1 de la clase minoritaria es la métrica clave aquí.

◆ QUIZ RÁPIDO

Tienes una columna 'talla' con valores S, M, L, XL. ¿Qué encoding usarías?

- A One-Hot Encoding, porque son categorías sin orden
- B Ordinal Encoding, porque hay un orden natural ($S < M < L < XL$) ✓**
- C Label Encoding, porque son texto
- D Ninguno, los modelos pueden manejar texto directamente

Respuesta — B. $S < M < L < XL$ tiene un orden semántico claro. Ordinal Encoding preserva esa relación numérica ($S=0, M=1, L=2, XL=3$). One-Hot Encoding ignoraría el orden y crearía 4 columnas innecesariamente.

◆ QUIZ RÁPIDO

Un modelo de fraude tiene Accuracy=99% en un dataset con 99% de transacciones legítimas. ¿Qué está pasando probablemente?

A El modelo es excelente y está perfectamente calibrado

B El modelo puede estar prediciendo siempre "legítima" — una accuracy engañosa por el desbalance ✓

C El modelo tiene data leakage del test set

D El dataset tiene demasiados outliers que el modelo maneja bien

Respuesta — B. En datasets 99/1, un modelo que siempre predice la clase mayoritaria tiene 99% accuracy sin aprender nada. Siempre usa F1, Recall o ROC-AUC en clasificación desbalanceada.

✓ MÓDULO COMPLETADO

Ahora puedes manejar los tres problemas más comunes en datos reales: variables categóricas, outliers y desbalance de clases. En el siguiente módulo añadirás *regularización* a tus modelos de regresión para combatir el overfitting con alta dimensionalidad.

// LIBRO II — TÉCNICAS AVANZADAS · MÓDULO 02

· 40 MIN · COMBATIR EL OVERFITTING EN REGRESIÓN

Regularización.

Cuando el modelo memoriza en vez de aprender — cómo añadir una penalización que fuerza la generalización.

La regresión lineal ordinaria minimiza el error de predicción sin ninguna restricción. Con muchas features, puede asignar coeficientes enormes a variables irrelevantes y memorizar el ruido del training set. La **regularización** añade una penalización al tamaño de los coeficientes, obligando al modelo a ser más conservador.

Ridge, Lasso y ElasticNet

Las tres variantes difieren en *cómo* penalizan los coeficientes. La diferencia clave: Lasso puede llevar coeficientes exactamente a cero (selección de features automática), Ridge los reduce pero nunca los elimina.

#	modelo	penalización	efecto	cuándo
0	Ridge (L2)	Suma de cuadrados de coeficientes	Reduce todos los coefs., ninguno llega a 0	Muchas features correlacionadas
1	Lasso (L1)	Suma de valores absolutos	Elimina features irrelevantes (coef.=0)	Muchas features, pocas relevantes
2	ElasticNet	Combinación L1 + L2	Intermedio: elimina algunas, reduce otras	Cuando no sabes cuál aplicar

Comparativa de métodos de regularización

EJERCICIO 1

· fácil

Efecto visual de la regularización en los coeficientes

Visualiza cómo Ridge, Lasso y ElasticNet afectan a los coeficientes comparados con regresión lineal sin regularización. Observa qué modelo elimina más features.

PYTHON

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.datasets import make_regression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

np.random.seed(42)

# Dataset con 20 features, solo 5 son realmente informativas
X, y = make_regression(n_samples=150, n_features=20, n_informative=5,
                      noise=30, random_state=42)

scaler = StandardScaler()
X_s = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_s, y,
                                                    test_size=0.2,
                                                    random_state=42)

modelos = {
    'LinearRegression': LinearRegression(),
    'Ridge (α=10)': Ridge(alpha=10),
    'Lasso (α=1)': Lasso(alpha=1),
    'ElasticNet': ElasticNet(alpha=1, l1_ratio=0.5),
}

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()

for ax, (nombre, modelo) in zip(axes, modelos.items()):
    modelo.fit(X_train, y_train)
    r2_train = modelo.score(X_train, y_train)
    r2_test = modelo.score(X_test, y_test)
    n_cero = np.sum(np.abs(modelo.coef_) < 0.01)

    ax.bar(range(20), modelo.coef_, color=['#C99419' if abs(coef) > 0.01 else 'black'],
           width=0.8)
    ax.axhline(0, color='black', linewidth=0.8)
    ax.set_title(f"{nombre}\nR² train={r2_train:.3f}, test={r2_test:.3f}")
    ax.set_xlabel('Feature'); ax.set_ylabel('Coeficiente')

plt.suptitle('Efecto de la regularización sobre los coeficientes')

```

```
plt.tight_layout()  
plt.show()
```

Pista: Lasso con alpha suficiente lleva muchos coeficientes exactamente a 0.0. Compara cuántas features "amarillas" (activas) quedan en cada modelo.

Selección automática de alpha con validación cruzada

Alpha (λ) controla la fuerza de la regularización. Alpha=0 es regresión lineal pura; alpha muy alto hace el modelo demasiado simple. scikit-learn ofrece `RidgeCV` y `LassoCV` que prueban múltiples valores automáticamente usando validación cruzada.

EJERCICIO 2

· intermedio

Selección automática de alpha — consumo energético

Aplica Ridge y Lasso con selección automática de alpha en un dataset de consumo energético con features de ruido. Verifica que Lasso elimina las variables irrelevantes.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import RidgeCV, LassoCV, LinearRe
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

np.random.seed(42)
n = 300

# Dataset simulado: consumo energético de edificios
superficie = np.random.normal(150, 60, n).clip(30, 500)
habitaciones = np.random.randint(1, 7, n)
antiguedad = np.random.randint(0, 50, n)
aislamiento = np.random.choice([0, 1, 2], n) # 0=ninguno,
orientacion = np.random.choice([0, 1], n) # 0=norte, 1
temp_media = np.random.normal(15, 5, n)

# Features adicionales irrelevantes (ruido)
ruido1 = np.random.randn(n)
ruido2 = np.random.randn(n)
ruido3 = np.random.randn(n)

consumo = (
    2.5 * superficie +
    80 * habitaciones +
    1.2 * antiguedad -
    400 * aislamiento -
    120 * orientacion -
    30 * temp_media +
    np.random.normal(0, 150, n)
)

feature_names = ['superficie', 'habitaciones', 'antiguedad']
X = np.column_stack([superficie, habitaciones, antiguedad,
                    ruido1, ruido2, ruido3])

scaler = StandardScaler()
X_s = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_s, co
```

```

alphas = np.logspace(-2, 4, 50)

ridge_cv = RidgeCV(alphas=alphas, cv=10).fit(X_train, y_train)
lasso_cv = LassoCV(alphas=alphas, cv=10, max_iter=5000).fit(X_train, y_train)
lr = LinearRegression().fit(X_train, y_train)

for nombre, modelo in [('LinearRegression', lr), ('RidgeCV', ridge_cv), ('LassoCV', lasso_cv)]:
    rmse_tr = mean_squared_error(y_train, modelo.predict(X_train))
    rmse_te = mean_squared_error(y_test, modelo.predict(X_test))
    alpha_str = f" α={modelo.alpha_:.3f}" if hasattr(modelo, 'alpha_') else ""
    print(f"{nombre}{alpha_str}")
    print(f" RMSE train={rmse_tr:.1f}, test={rmse_te:.1f}")

# Comparar coeficientes
fig, axes = plt.subplots(1, 3, figsize=(14, 5))
for ax, (nombre, modelo) in zip(axes, [('LinearRegression', lr), ('RidgeCV', ridge_cv), ('LassoCV', lasso_cv)]):
    colores = ['#C99419' if abs(c) > 0.01 else '#bbb' for c in modelo.coef_]
    ax.barh(feature_names, modelo.coef_, color=colores)
    ax.axvline(0, color='black', linewidth=0.8)
    ax.set_title(nombre)
plt.suptitle('Coeficientes estandarizados', y=1.02)
plt.tight_layout()
plt.show()

```

Pista: RidgeCV y LassoCV exponen `.alpha_` con el `alpha` óptimo encontrado. `np.logspace(-2, 4)` crea una escala logarítmica de 0.01 a 10000.

◆ QUIZ RÁPIDO

¿Cuál es la principal diferencia entre Ridge y Lasso?

- A Ridge es más rápido de calcular que Lasso
- B** Lasso puede llevar coeficientes exactamente a cero (selección de features), Ridge solo los reduce ✓
- C Ridge funciona solo para clasificación, Lasso para regresión
- D Lasso usa validación cruzada automáticamente, Ridge no

Respuesta — B. La penalización L1 de Lasso genera soluciones sparse: muchos coeficientes exactamente 0. Ridge (L2) reduce los coeficientes pero raramente los elimina. Por eso Lasso actúa como selector automático de features.

◆ QUIZ RÁPIDO

Tu modelo de regresión tiene $R^2=0.97$ en training y $R^2=0.61$ en test. ¿Qué haría regularización?

- ☐ A Empeoraría ambos scores porque añade restricciones innecesarias
- ☒ B Reduciría el R^2 de training ligeramente pero mejoraría el de test al reducir overfitting ✓
- ☐ C Aumentaría el R^2 de training porque el modelo aprende mejor
- ☐ D Solo afecta al training set, no al test

Respuesta — B. 0.97 en train y 0.61 en test es overfitting claro. La regularización penaliza coeficientes grandes, forzando al modelo a encontrar patrones más generalizables. El train score baja un poco, el test score sube — ese es el objetivo.

✓ MÓDULO COMPLETADO

Ya sabes combatir el overfitting con regularización y seleccionar automáticamente el hiperparámetro alpha. En el siguiente módulo explorarás las *Máquinas de Soporte Vectorial*— un algoritmo diferente que maximiza el margen de separación entre clases.

// LIBRO II — TÉCNICAS AVANZADAS · MÓDULO 03

· 45 MIN · EL CLASIFICADOR DE MÁXIMO MARGEN

Máquinas de *Soporte Vectorial.*

SVM no busca cualquier línea que separe las clases — busca la que las separa con el mayor margen posible.

Las **Máquinas de Soporte Vectorial (SVM)** son uno de los algoritmos más elegantes del ML clásico. Mientras que la regresión logística busca una frontera que clasifique correctamente, SVM busca la frontera que maximiza la distancia a los puntos más cercanos de cada clase — los llamados *vectores de soporte*.

Esta diferencia parece pequeña, pero tiene grandes consecuencias: SVM tiende a generalizar mejor porque la frontera está lo más alejada posible de ambas clases. Además, con el **kernel trick**, SVM puede clasificar datos que no son linealmente separables proyectándolos a dimensiones superiores.

Kernels: la clave de la flexibilidad

#	kernel	descripción	cuándo
0	linear	Frontera lineal (hiperplano)	Datos linealmente separables, muchas features
1	rbf	Radial Basis Function – frontera circular/curva	Datos no lineales, default recomendado
2	poly	Frontera polinómica de grado n	Relaciones polinómicas conocidas

Kernels principales en SVM y cuándo usarlos

EJERCICIO 1

· fácil

SVM con diferentes kernels – comparativa visual

Visualiza cómo los diferentes kernels generan fronteras de decisión distintas en dos datasets clásicos no lineales.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.datasets import make_moons, make_circles
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score

np.random.seed(42)

# Datos en forma de lunas: no linealmente separables
X_moons, y_moons = make_moons(n_samples=200, noise=0.2, random_state=42)
X_circles, y_circles = make_circles(n_samples=200, noise=0.1, random_state=42)

scaler = StandardScaler()

fig, axes = plt.subplots(2, 3, figsize=(13, 8))

datasets = [('Lunas', X_moons, y_moons), ('Círculos', X_circles, y_circles)]
kernels = ['linear', 'rbf', 'poly']

for row, (ds_name, X_raw, y) in enumerate(datasets):
    X = scaler.fit_transform(X_raw)
    for col, kernel in enumerate(kernels):
        ax = axes[row, col]
        svm = SVC(kernel=kernel, C=1.0, gamma='scale')
        svm.fit(X, y)

        # Generar grid para visualizar frontera
        xx, yy = np.meshgrid(np.linspace(X[:,0].min()-0.5, X[:,0].max()+0.5, 10),
                             np.linspace(X[:,1].min()-0.5, X[:,1].max()+0.5, 10))
        Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
        ax.contourf(xx, yy, Z, alpha=0.3, cmap='RdYlBu')
        ax.scatter(X[:,0], X[:,1], c=y, cmap='RdYlBu', edgecolors='k')

        acc = cross_val_score(svm, X, y, cv=5).mean()
        ax.set_title(f"{ds_name} | kernel={kernel}\nAcc={acc:.2f}")
        ax.set_xticks([]); ax.set_yticks([])

plt.suptitle('SVM – Fronteras de decisión según kernel', fontweight='bold', y=1.05)
```

```
plt.tight_layout()  
plt.show()
```

Pista: El kernel rbf es el más flexible y funciona bien para la mayoría de problemas no lineales. El kernel lineal falla con datos en forma de luna o círculos concéntricos.

El hiperparámetro C – margen duro vs suave

C controla el **tradeoff entre margen y errores de clasificación**: C pequeño → margen más amplio pero acepta algunos errores (más robusto al ruido). C grande → intenta clasificar todo correctamente aunque el margen sea estrecho (más susceptible a overfitting).

EJERCICIO 2

· intermedio

Clasificador de calidad de vino con SVM

Clasifica 3 tipos de vino por sus propiedades químicas. Explora el efecto de C y usa GridSearchCV para encontrar los mejores hiperparámetros automáticamente.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.metrics import classification_report, ConfusionMatrixDisplay

# Dataset de vinos: 3 clases, 13 features químicas
datos = load_wine()
X, y = datos.data, datos.target
nombres_clases = datos.target_names

scaler = StandardScaler()
X_s = scaler.fit_transform(X)

# Explorar efecto de C
C_valores = [0.01, 0.1, 1, 10, 100, 1000]
accuracies = []
for C in C_valores:
    svm = SVC(kernel='rbf', C=C, gamma='scale')
    acc = cross_val_score(svm, X_s, y, cv=5).mean()
    accuracies.append(acc)
    print(f"C={C:6}: accuracy={acc:.4f}")

print()

# Grid search para encontrar mejores parámetros
param_grid = {'C': [0.1, 1, 10, 100], 'gamma': ['scale', 'auto']}
gs = GridSearchCV(SVC(kernel='rbf'), param_grid, cv=5, scoring='accuracy')
gs.fit(X_s, y)
print(f"Mejores params: {gs.best_params_}")
print(f"Mejor accuracy: {gs.best_score_:.4f}")

# Visualizar efecto de C
plt.figure(figsize=(8, 4))
plt.semilogx(C_valores, accuracies, 'o-', color='#1F4E5F',
             label='C (escala log)')
plt.ylabel('Accuracy (CV 5-fold)')
```



```
plt.title('Efecto del hiperparámetro C en SVM con kernel RB')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Pista: GridSearchCV prueba todas las combinaciones de C y gamma. `.best_params_` devuelve los parámetros óptimos. Nota cómo el accuracy forma una curva de campana con C.

◆ QUIZ RÁPIDO

¿Qué son los 'vectores de soporte' en SVM?

- A Todos los puntos del dataset de entrenamiento
- B Los puntos más cercanos a la frontera de decisión que definen el margen ✓**
- C Los vectores de gradiente calculados durante el entrenamiento
- D Los puntos mal clasificados por el modelo

Respuesta — B. Los vectores de soporte son los puntos de cada clase más cercanos a la frontera de decisión. Son literalmente los únicos puntos que importan para definir la frontera — si eliminas cualquier otro punto del training set, el modelo no cambiaría.

◆ QUIZ RÁPIDO

¿Cuándo deberías usar `kernel='linear'` en lugar de `'rbf'`?

A Cuando tienes muchos datos (>10000 muestras) porque RBF es demasiado lento

B Cuando tienes muchas features y los datos son aproximadamente linealmente separables ✓

C Cuando quieres visualizar la frontera de decisión en 2D

D Linear siempre es mejor porque es más simple

Respuesta — B. El kernel lineal es eficiente con muchas features (como en clasificación de texto con TF-IDF). En alta dimensión, los datos suelen ser linealmente separables. RBF es más poderoso pero puede hacer overfitting con pocas muestras y muchas features.

✓ MÓDULO COMPLETADO

SVM es una herramienta poderosa especialmente útil con datos de alta dimensión o cuando el número de muestras no es enorme. En el siguiente módulo explorarás el aprendizaje *no supervisado* — clustering — donde no hay etiquetas y el modelo encuentra la estructura por sí solo.

// LIBRO II — TÉCNICAS AVANZADAS · MÓDULO 04

· 40 MIN · APRENDIZAJE SIN ETIQUETAS

Clustering.

¿Qué pasa cuando no tienes etiquetas? El modelo encuentra la estructura oculta de los datos por sí solo.

Hasta ahora todos los modelos que hemos usado son de **aprendizaje supervisado**: tienes datos y la respuesta correcta para cada uno. Pero ¿qué pasa cuando solo tienes datos sin etiquetas? Eso es el **aprendizaje no supervisado**.

El clustering agrupa puntos similares sin saber de antemano qué grupos existen. Es útil para segmentar clientes, detectar anomalías, agrupar noticias similares o entender la estructura natural de un dataset antes de etiquetarlo.

K-Means: el algoritmo más popular

K-Means funciona iterativamente: elige K centroides al azar, asigna cada punto al centroide más cercano, recalcula los centroides como la media de su grupo, y repite hasta converger. La dificultad: debes especificar K de antemano.

EJERCICIO 1

· fácil

K-Means y el método del codo

Aplica K-Means con el método del codo para encontrar el número óptimo de clusters. Compara los clusters encontrados con la estructura real de los datos.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_blobs

np.random.seed(42)

# Crear 4 clusters bien diferenciados
X, y_real = make_blobs(n_samples=300, centers=4, cluster_std=0.6)
scaler = StandardScaler()
X_s = scaler.fit_transform(X)

# Método del codo: encontrar K óptimo
inertias = []
K_range = range(1, 11)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_s)
    inertias.append(km.inertia_)

# Encontrar el codo automáticamente
diffs = np.diff(inertias)
diffs2 = np.diff(diffs)
codo_k = np.argmax(diffs2) + 2 # +2 por doble diff y 0-index

print(f"K óptimo sugerido por el método del codo: {codo_k}")
print()

# Ajustar con el K óptimo
km_final = KMeans(n_clusters=codo_k, random_state=42, n_init=10)
etiquetas = km_final.fit_predict(X_s)

fig, axes = plt.subplots(1, 3, figsize=(14, 4))

# Método del codo
axes[0].plot(K_range, inertias, 'o-', color='#1F4E5F', line
axes[0].axvline(codo_k, color='#C99419', linestyle='--', la
axes[0].set_xlabel('Número de clusters K')
```

```
axes[0].set_ylabel('Inercia (WCSS)')
axes[0].set_title('Método del codo')
axes[0].legend(); axes[0].grid(True, alpha=0.3)

# Clusters encontrados
sc = axes[1].scatter(X_s[:,0], X_s[:,1], c=etiquetas, cmap=
axes[1].scatter(km_final.cluster_centers_[0], km_final.cl
                    marker='X', s=200, c='black', zorder=5, lab
axes[1].set_title(f'K-Means con K={codo_k}'); axes[1].legen

# Clusters reales
axes[2].scatter(X_s[:,0], X_s[:,1], c=y_real, cmap='tab10',
axes[2].set_title('Grupos reales (referencia)')

plt.tight_layout()
plt.show()
```

Pista: La inercia mide la suma de distancias al cuadrado de cada punto a su centroide. El "codo" es donde agregar más clusters ya no reduce mucho la inercia.

DBSCAN: clustering sin especificar K

DBSCAN (Density-Based Spatial Clustering) encuentra clusters como zonas densas separadas por regiones de baja densidad. Ventajas clave: no necesitas especificar K, puede encontrar clusters de forma arbitraria, y detecta outliers automáticamente (puntos sin cluster).

EJERCICIO 2

· intermedio

Segmentación de clientes con K-Means y DBSCAN

Segmenta clientes con K-Means y DBSCAN. Compara los perfiles resultantes y usa el Silhouette Score para evaluar la calidad del clustering.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

np.random.seed(42)
n = 400

# Dataset de clientes: [gasto_mensual, frecuencia_compras,
gasto = np.concatenate([
    np.random.normal(200, 50, 100), # clientes básicos
    np.random.normal(800, 100, 100), # clientes premium
    np.random.normal(400, 80, 150), # clientes medios
    np.random.normal(50, 20, 50), # clientes inactivos
])

frecuencia = np.concatenate([
    np.random.normal(2, 0.5, 100),
    np.random.normal(8, 1.2, 100),
    np.random.normal(4, 1.0, 150),
    np.random.normal(0.5, 0.2, 50),
]).clip(0)

antiguedad = np.concatenate([
    np.random.normal(12, 4, 100),
    np.random.normal(36, 8, 100),
    np.random.normal(18, 6, 150),
    np.random.normal(6, 3, 50),
]).clip(1)

X = np.column_stack([gasto, frecuencia, antiguedad])
scaler = StandardScaler()
X_s = scaler.fit_transform(X)

# K-Means con K=4
km = KMeans(n_clusters=4, random_state=42, n_init=10)
km_labels = km.fit_predict(X_s)
km_sil = silhouette_score(X_s, km_labels)

# DBSCAN
```



```

db = DBSCAN(eps=0.5, min_samples=10)
db_labels = db.fit_predict(X_s)
n_clusters_db = len(set(db_labels)) - (1 if -1 in db_labels
n_outliers = (db_labels == -1).sum()
db_sil = silhouette_score(X_s, db_labels) if n_clusters_db

print(f"K-Means: {len(set(km_labels))} clusters, silhouette
print(f"DBSCAN: {n_clusters_db} clusters, {n_outliers} out
print()
print("Perfil de clientes K-Means:")
feature_names = ['Gasto mensual', 'Frecuencia', 'Antigüedad
for k in range(4):
    mask = km_labels == k
    centro = scaler.inverse_transform(km.cluster_centers_)
    print(f" Cluster {k} ({mask.sum()} clientes): gasto={c

# Visualización
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, (labels, titulo) in zip(axes,
    [(km_labels, f'K-Means (K=4, sil={km_sil:.2f})'),
     (db_labels, f'DBSCAN (eps=0.5, sil={db_sil:.2f})')]):
    sc = ax.scatter(gasto, frecuencia, c=labels, cmap='tab1
    ax.set_xlabel('Gasto mensual (FCFA)'); ax.set_ylabel('F
    ax.set_title(titulo)
plt.tight_layout()
plt.show()

```

Pista: El Silhouette Score mide la calidad del clustering (-1 a 1, mayor es mejor). En DBSCAN, los puntos con label=-1 son outliers no asignados a ningún cluster.

◆ QUIZ RÁPIDO

¿Cuál es la principal limitación de K-Means?

- A Es demasiado lento para datasets grandes
- B Debes especificar K de antemano y asume clusters esféricos de tamaño similar ✓**
- C No puede trabajar con datos numéricos continuos
- D Solo funciona con 2 features

Respuesta — B. K-Means requiere que especifiques K antes de correr, y asume que los clusters son aproximadamente esféricos y de tamaño similar. DBSCAN resuelve el primer problema (descubre K automáticamente) y puede encontrar clusters de forma arbitraria.

◆ QUIZ RÁPIDO

¿Qué representa un punto con `label=-1` en DBSCAN?

- A Un punto que pertenece al primer cluster (índice -1)
- B Un outlier: un punto demasiado aislado para pertenecer a ningún cluster ✓**
- C Un error del algoritmo que indica que los parámetros son incorrectos
- D El centroide del cluster más cercano

Respuesta — B. En DBSCAN, `label=-1` significa 'ruido' o outlier — un punto que no tiene suficientes vecinos cercanos para formar parte de un cluster. Esta detección automática de anomalías es una de las grandes ventajas de DBSCAN.

✓ MÓDULO COMPLETADO

El clustering te permite explorar datos sin etiquetar y descubrir su estructura natural. En el siguiente módulo aprenderás *PCA* — cómo reducir cientos de dimensiones a unas pocas sin perder la información esencial.

// LIBRO II — TÉCNICAS AVANZADAS · MÓDULO 05

· 40 MIN · VER LO INVISIBLE EN ALTA DIMENSIÓN

PCA y reducción de dimensiones.

Con 500 features no puedes visualizar nada. PCA comprime la información esencial en 2 o 3 dimensiones.

Muchos datasets reales tienen decenas o cientos de features. Esto crea el **problema de la maldición de la dimensionalidad**: con más dimensiones, los puntos se vuelven más dispersos, los modelos necesitan exponencialmente más datos y la visualización es imposible.

El **Análisis de Componentes Principales (PCA)** encuentra las direcciones del espacio que contienen la mayor varianza en los datos y proyecta todo ahí. No es simplemente elegir las mejores features — es crear *nuevas* features que son combinaciones lineales de las originales, maximizando la varianza capturada.

Cómo funciona PCA

Cada **componente principal** es una dirección ortogonal que captura la máxima varianza restante después de las componentes anteriores. La primera captura más varianza que cualquier otra dirección; la segunda captura la máxima del espacio ortogonal a la primera; etc.

EJERCICIO 1

· fácil

PCA paso a paso — varianza explicada

Analiza cuántas componentes necesitas para capturar el 95% y 99% de la varianza en un dataset de cáncer de mama con 30 features.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler

# Dataset de cáncer de mama: 30 features → ¿cuántas necesit
datos = load_breast_cancer()
X, y = datos.data, datos.target
nombres = datos.feature_names

scaler = StandardScaler()
X_s = scaler.fit_transform(X)

# PCA completo para analizar varianza
pca_full = PCA()
pca_full.fit(X_s)

varianza_acum = np.cumsum(pca_full.explained_variance_ratio
n_95 = np.argmax(varianza_acum ≥ 0.95) + 1
n_99 = np.argmax(varianza_acum ≥ 0.99) + 1

print(f"Features originales: {X_s.shape[1]}")
print(f"Componentes para 95% de varianza: {n_95}")
print(f"Componentes para 99% de varianza: {n_99}")
print()
print("Varianza explicada por primeras 10 PCs:")
for i, v in enumerate(pca_full.explained_variance_ratio[:1
    barra = '█' * int(v * 100)
    print(f"  PC{i+1:2d}: {v:.3f}  {barra}")

# Visualización
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Varianza por componente
axes[0].bar(range(1, 16), pca_full.explained_variance_ratio
            color='#1F4E5F', alpha=0.8)
axes[0].set_xlabel('Componente principal')
axes[0].set_ylabel('% de varianza explicada')
```

```
axes[0].set_title('Varianza por componente (primeras 15)')

# Varianza acumulada
axes[1].plot(range(1, 31), varianza_acum * 100, 'o-', color=
axes[1].axhline(95, color='#C99419', linestyle='--', label=
axes[1].axhline(99, color='#A8331E', linestyle='--', label=
axes[1].axvline(n_95, color='#C99419', linestyle=':', alpha=
axes[1].axvline(n_99, color='#A8331E', linestyle=':', alpha=
axes[1].set_xlabel('Número de componentes')
axes[1].set_ylabel('% de varianza acumulada')
axes[1].set_title('Varianza acumulada')
axes[1].legend(); axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Pista: explained_variance_ratio_ es un array con la fracción de varianza explicada por cada componente. np.cumsum() calcula la suma acumulada.

PCA para visualización 2D

Una de las aplicaciones más valiosas de PCA: comprimir datos de alta dimensión a 2 dimensiones para visualizarlos. Aunque perdemos información, las 2 primeras componentes suelen capturar la estructura más importante.

EJERCICIO 2

· intermedio

Visualización y compresión con PCA

Usa PCA para visualizar 1797 dígitos manuscritos en 2D y compara el accuracy de KNN con diferentes niveles de compresión. ¿Cuántas componentes dan el mejor resultado?

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

# Dataset de dígitos: 64 features (8×8 píxeles), 10 clases
digits = load_digits()
X, y = digits.data, digits.target

scaler = StandardScaler()
X_s = scaler.fit_transform(X)

# KNN con todas las features
knn_full = KNeighborsClassifier(n_neighbors=5)
acc_full = cross_val_score(knn_full, X_s, y, cv=5).mean()
print(f"KNN con 64 features: accuracy={acc_full:.3f}")

# PCA a diferentes dimensiones
dims = [2, 5, 10, 20, 30, 40]
accuracies = []
for n in dims:
    pca = PCA(n_components=n)
    X_pca = pca.fit_transform(X_s)
    knn = KNeighborsClassifier(n_neighbors=5)
    acc = cross_val_score(knn, X_pca, y, cv=5).mean()
    var = pca.explained_variance_ratio_.sum()
    accuracies.append(acc)
    print(f"PCA({n:2d}): varianza={var:.1%}, accuracy={acc:.3f}")

print()

# Proyección 2D para visualización
pca_2d = PCA(n_components=2)
X_2d = pca_2d.fit_transform(X_s)
var_2d = pca_2d.explained_variance_ratio_.sum()
print(f"Varianza capturada con 2 componentes: {var_2d:.1%}")
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Scatter 2D de los dígitos
scatter = axes[0].scatter(X_2d[:,0], X_2d[:,1], c=y, cmap='
plt.colorbar(scatter, ax=axes[0])
axes[0].set_title(f'Dígitos en 2D con PCA ({var_2d:.0%} var
axes[0].set_xlabel('PC1'); axes[0].set_ylabel('PC2')

# Accuracy vs número de componentes
axes[1].plot(dims, accuracies, 'o-', color='#1F4E5F', linewidth
axes[1].axhline(acc_full, color='#C99419', linestyle='--',
axes[1].set_xlabel('Número de componentes PCA')
axes[1].set_ylabel('Accuracy (CV 5-fold)')
axes[1].set_title('Accuracy vs compresión PCA')
axes[1].legend(); axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Pista: Con PCA(n_components=20) sobre 64 features, el KNN suele mejorar su accuracy (menos ruido) además de ser más rápido. Observa el "codo" en la curva accuracy vs componentes.

◆ QUIZ RÁPIDO

¿Por qué es importante normalizar los datos ANTES de aplicar PCA?

A PCA falla si los datos no están normalizados

B Porque PCA se basa en varianza, y features con mayor escala dominarán artificialmente las componentes ✓

C Para que PCA sea más rápido computacionalmente

D Porque la normalización elimina las correlaciones entre features

Respuesta — B. PCA maximiza la varianza. Si 'salario' va de 0 a 100.000 y 'edad' de 0 a 100, la varianza de salario dominará las componentes aunque edad sea más informativa. StandardScaler pone todas las features en la misma escala antes de que PCA las compare.

◆ QUIZ RÁPIDO

¿Qué significa que las dos primeras componentes de PCA explican el 85% de la varianza?

A El 85% de las features originales están representadas en esas dos componentes

B El 85% de la información (variabilidad) del dataset puede representarse en 2 dimensiones ✓

C El modelo entrenado en esas 2 dimensiones tendrá 85% de accuracy

D El 15% de los datos son outliers que PCA ignoró

Respuesta — B. La varianza explicada mide qué proporción de la variabilidad total del dataset está capturada en las componentes. 85% significa que proyectar a 2D preserva el 85% de la 'información' (dispersión) de los datos originales.

✓ MÓDULO COMPLETADO

PCA es una herramienta fundamental para visualización, reducción de ruido y como paso previo a otros modelos. En el siguiente módulo aprenderás técnicas de *selección de features*— cómo elegir las variables más relevantes basándote en estadísticas y en los propios modelos.

// LIBRO II — TÉCNICAS AVANZADAS · MÓDULO 06

· 35 MIN · MENOS FEATURES, MEJOR MODELO

Selección de *características*.

Más features no siempre es mejor. Elegir las correctas reduce overfitting, acelera el entrenamiento y mejora la interpretabilidad.

Tener 200 features cuando solo 15 son relevantes perjudica al modelo: introduce ruido, genera correlaciones espurias y hace el modelo más lento e interpretable. La **selección de features** es el proceso de identificar y conservar solo las variables más útiles.

Hay tres familias de métodos: *filter methods* (estadísticos, independientes del modelo), *wrapper methods* (prueban subconjuntos de features con el modelo) y *embedded methods* (el propio modelo selecciona, como Lasso o árboles).

#	método	familia	idea	pros
0	Varianza	Filter	Elimina features con muy poca varianza (casi constantes)	Muy rápido
1	Correlación / ANOVA	Filter	Selecciona features más correlacionadas con el target	Independiente del modelo
2	SelectFromModel	Embedded	Usa importancias de un árbol/Lasso para filtrar	Captura interacciones no lineales
3	RFE	Wrapper	Elimina la feature menos importante iterativamente	Muy preciso pero costoso

Métodos principales de selección de features

EJERCICIO 1

· fácil

Comparando métodos de selección de features

Compara 5 métodos de selección de features en un dataset con features de ruido. Observa cómo reducir features puede mejorar (o mantener) la accuracy.

PYTHON

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.feature_selection import (VarianceThreshold, SelectFromModel, RF
                                     SelectFromModel, RF
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LassoCV
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

# Dataset: 5 features informativas, 15 de ruido
X, y = make_classification(
    n_samples=300, n_features=20, n_informative=5,
    n_redundant=5, n_repeated=0, random_state=42
)

# Añadir 2 features casi constantes
X[:, 18] = 0.01 + np.random.randn(300) * 0.001
X[:, 19] = 5.0 + np.random.randn(300) * 0.001

scaler = StandardScaler()
X_s = scaler.fit_transform(X)

rf_eval = RandomForestClassifier(n_estimators=100, random_s

metodos = {
    'Todas (20)': X_s,
    'Varianza (umbral)': VarianceThreshold(threshold=0.01).
    'ANOVA top-10': SelectKBest(f_classif, k=10).fit_t
    'RF importances': SelectFromModel(RandomForestClassi
    'Lasso': SelectFromModel(LassoCV(cv=5, max_
    'RFE (10 feat.)': RFE(RandomForestClassifier(n_estim
}

print(f"{'Método':<20} {'Features':>8} {'Accuracy':>10}")
print('-' * 42)
resultados = {}
for nombre, X_sel in metodos.items():

```



```
acc = cross_val_score(rf_eval, X_sel, y, cv=5).mean()
resultados[nombre] = (X_sel.shape[1], acc)
print(f"{nombre:<20} {X_sel.shape[1]:>8} {acc:>10.3f}")

# Gráfico
fig, ax = plt.subplots(figsize=(10, 5))
nombres = list(resultados.keys())
features = [resultados[n][0] for n in nombres]
accuracies = [resultados[n][1] for n in nombres]
colores = ['#ccc' if n == 'Todas (20)' else '#1F4E5F' for n in nombres]

ax.barh(nombres, accuracies, color=colores, alpha=0.8)
for i, (f, a) in enumerate(zip(features, accuracies)):
    ax.text(a + 0.001, i, f'{a:.3f} ({f} feat.)', va='center')
ax.set_xlabel('Accuracy (CV 5-fold)')
ax.set_title('Comparativa de métodos de selección de features')
ax.set_xlim(0.7, 1.0)
plt.tight_layout()
plt.show()
```

Pista: SelectFromModel con un Random Forest usa las `feature_importances_` del modelo para decidir qué conservar. El umbral por defecto es la media de importancias.

Análisis de importancia con Random Forest

EJERCICIO 2

· intermedio

Importancia de features en datos de salud

Compara la importancia por impureza (MDI) vs importancia por permutación en datos de diabetes. Observa si ambos métodos coinciden en las features más relevantes.

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

datos = load_diabetes()
X, y = datos.data, datos.target
feature_names = datos.feature_names

scaler = StandardScaler()
X_s = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_s, y,
                                                    test_size=0.3,
                                                    random_state=42)

rf = RandomForestRegressor(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)

# Importancia por impureza (built-in) – rápida pero puede ser poco fiable
imp_builtin = rf.feature_importances_

# Importancia por permutación – más fiable
perm = permutation_importance(rf, X_test, y_test, n_repeats=10,
                              random_state=42)
imp_perm_mean = perm.importances_mean
imp_perm_std = perm.importances_std

print(f"R² test: {rf.score(X_test, y_test):.3f}")
print()
print(f"{'Feature':<12} {'Imp. impureza':>14} {'Imp. permutación':>14}")
orden = np.argsort(imp_perm_mean)[::-1]
for i in orden:
    print(f"{feature_names[i]:<12} {imp_builtin[i]:>14.4f} {imp_perm_std[i]:>14.4f}")

# Visualización
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
orden_builtin = np.argsort(imp_builtin)
axes[0].barh([feature_names[i] for i in orden_builtin],
              [imp_builtin[i] for i in orden_builtin], color='blue')
```

```
axes[0].set_title('Importancia por impureza (MDI)')
axes[0].set_xlabel('Importancia')

orden_perm = np.argsort(imp_perm_mean)
axes[1].barh([feature_names[i] for i in orden_perm],
             [imp_perm_mean[i] for i in orden_perm],
             xerr=[imp_perm_std[i] for i in orden_perm],
             color='#C99419', alpha=0.8, capsize=4)
axes[1].set_title('Importancia por permutación')
axes[1].set_xlabel('Disminución en R²')

plt.suptitle('Comparativa de métodos de importancia', y=1.0)
plt.tight_layout()
plt.show()
```

Pista: La importancia por permutación mezcla aleatoriamente cada feature y mide cuánto baja el R^2 . Es más fiable que la importancia MDI, especialmente con features correlacionadas.

◆ QUIZ RÁPIDO

¿Por qué la importancia por permutación es más fiable que la importancia MDI (impureza) de Random Forest?

- A Porque es más rápida de calcular
- B Porque mide el impacto real en el conjunto de test, evitando el sesgo de MDI hacia features de alta cardinalidad o numéricas ✓**
- C Porque MDI solo funciona con clasificación, no regresión
- D Porque permutación usa todos los datos mientras MDI usa solo el training set

Respuesta — B. MDI (Mean Decrease in Impurity) se calcula en training y tiende a sobreestimar la importancia de features con muchos valores únicos (alta cardinalidad). La importancia por permutación se mide en test y refleja el impacto real en nuevos datos.

◆ QUIZ RÁPIDO

Tienes un dataset con 500 features y 200 muestras. ¿Qué harías primero antes de entrenar?

A Entrenar directamente con todas las features — más es mejor

B Aplicar selección de features o PCA para reducir dimensionalidad antes de entrenar ✓

C Aumentar el número de muestras con data augmentation

D Usar solo las 10 primeras features del dataset

Respuesta — B. Con 500 features y 200 muestras ($n \ll p$), cualquier modelo sobreajustará trivialmente. Selección de features o PCA es el primer paso obligatorio. Esta situación (más features que muestras) es común en bioinformática y análisis de texto.

✓ MÓDULO COMPLETADO

Ahora tienes un arsenal completo de técnicas de selección de features. En el siguiente módulo aprenderás a encadenar todos estos pasos — preprocesamiento, selección, escalado, modelo — en un único objeto llamado *Pipeline*.

// LIBRO II — TÉCNICAS AVANZADAS · MÓDULO 07

· 40 MIN · EL CÓDIGO QUE VA A PRODUCCIÓN

Pipelines.

Un Pipeline encadena todo el procesamiento en un objeto reproducible, sin data leakage y listo para producción.

En los módulos anteriores has aplicado escalado, encoding, selección de features y modelo como pasos separados. En la práctica esto es propenso a errores — especialmente el **data leakage**: si escales los datos antes del split, el modelo "ve" el test set.

La solución es el **Pipeline** de scikit-learn: un objeto que encadena todos los pasos de transformación y el modelo final. Cuando llamas `pipeline.fit(X_train)`, todos los pasos aprenden solo del training set. Cuando llamas `pipeline.predict(X_test)`, aplica las transformaciones aprendidas al test sin reaprender — garantizando que no hay leakage.

PYTHON

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC

# Pipeline: scaler → PCA → SVM
pipeline = Pipeline([
    ('scaler', StandardScaler()), # Paso 1: normalizar
    ('pca', PCA(n_components=10)), # Paso 2: reducir dimensiones
    ('svm', SVC(kernel='rbf', C=1.0)), # Paso 3: clasificar
])

# fit() solo en training set – sin data leakage posible
pipeline.fit(X_train, y_train)

# predict() aplica automáticamente todos los pasos al test
accuracy = pipeline.score(X_test, y_test)
print(f"Accuracy: {accuracy:.3f}")
```

ColumnTransformer: preprocesamiento mixto

En datos reales tienes columnas numéricas y categóricas que necesitan transformaciones distintas. `ColumnTransformer` aplica transformadores diferentes a diferentes columnas, y se puede usar dentro de un Pipeline.

EJERCICIO 1

· fácil

Pipeline completo con datos mixtos

Construye un pipeline completo con datos mixtos (numéricos + categóricos), valores nulos y un clasificador. El Pipeline garantiza que no hay data leakage en la validación cruzada.

PYTHON

```
import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score

np.random.seed(42)
n = 400

# Dataset realista: empleados con datos mixtos y valores nulos
df = pd.DataFrame({
    'edad': np.random.randint(22, 60, n).astype(float),
    'salario': np.random.normal(2500, 800, n).clip(800, 12000),
    'años_empresa': np.random.randint(0, 25, n).astype(float),
    'nivel_estudios': np.random.choice(['Primaria', 'Secundaria', 'Universitaria'], n),
    'departamento': np.random.choice(['Ventas', 'IT', 'RRHH', 'Marketing', 'Operaciones'], n),
    'horas_extra': np.random.normal(5, 3, n).clip(0, 20)
})

# Añadir valores nulos realistas
for col in ['edad', 'salario', 'años_empresa', 'horas_extra']:
    idx = np.random.choice(n, int(n * 0.08), replace=False)
    df.loc[idx, col] = np.nan

# Target: abandona la empresa (1) o no (0)
df['abandona'] = (
    (df['salario'].fillna(df['salario'].median()) < 1800) |
    (df['horas_extra'].fillna(0) > 10)
).astype(int)
flip = np.random.rand(n) < 0.1
df['abandona'] ^= flip

X = df.drop('abandona', axis=1)
y = df['abandona']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```

# Identificar columnas por tipo
cols_num = ['edad', 'salario', 'años_empresa', 'horas_extra']
cols_cat = ['nivel_estudios', 'departamento']

# Transformadores por tipo de columna
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('ohe', OneHotEncoder(handle_unknown='ignore', sparse=True)),
])

preprocessor = ColumnTransformer([
    ('num', num_pipeline, cols_num),
    ('cat', cat_pipeline, cols_cat),
])

# Pipeline completo
full_pipeline = Pipeline([
    ('prep', preprocessor),
    ('model', RandomForestClassifier(n_estimators=100, class_weight='balanced'))
])

full_pipeline.fit(X_train, y_train)
acc_test = full_pipeline.score(X_test, y_test)
cv_scores = cross_val_score(full_pipeline, X, y, cv=5)

print(f"Nulls en el dataset: {X.isnull().sum().sum()} valor")
print(f"Distribución target: {y.mean():.1%} abandonan")
print()
print(f"Accuracy test: {acc_test:.3f}")
print(f"Accuracy CV (5-fold): {cv_scores.mean():.3f} ± {cv_scores.std():.3f}")

```

Pista: ColumnTransformer aplica transformadores diferentes a subconjuntos de columnas y concatena los resultados.

`SimpleImputer(strategy="median")` rellena nulos con la mediana de la columna.

Búsqueda de hiperparámetros en el Pipeline

EJERCICIO 2

· intermedio

GridSearchCV sobre un Pipeline completo

Aplica GridSearchCV a un Pipeline completo (StandardScaler → PCA → Random Forest) para optimizar simultáneamente la compresión PCA y los hiperparámetros del modelo.

PYTHON

```
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, cross_val
from sklearn.datasets import load_breast_cancer

datos = load_breast_cancer()
X, y = datos.data, datos.target

# Pipeline: scaler → PCA → Random Forest
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA()),
    ('rf', RandomForestClassifier(random_state=42)),
])

# Los parámetros se nombran como 'paso__parametro'
param_grid = {
    'pca__n_components': [5, 10, 15, 20],
    'rf__n_estimators': [50, 100],
    'rf__max_depth': [None, 5, 10],
}

gs = GridSearchCV(
    pipeline, param_grid,
    cv=5, scoring='accuracy',
    n_jobs=-1, verbose=0
)
gs.fit(X, y)

print("Mejores parámetros:")
for k, v in gs.best_params_.items():
    print(f" {k}: {v}")
print()
print(f"Mejor accuracy CV: {gs.best_score_:.4f}")

# Comparar con pipeline sin optimización
```

```
pipeline_base = Pipeline([
    ('scaler', StandardScaler()),
    ('rf',      RandomForestClassifier(n_estimators=100, ran
])
acc_base = cross_val_score(pipeline_base, X, y, cv=5).mean()
print(f"Sin PCA ni tuning:  {acc_base:.4f}")
print()

# Top 5 configuraciones
import pandas as pd
resultados = pd.DataFrame(gs.cv_results_)
top5 = resultados.nlargest(5, 'mean_test_score')[['param_pc
print("Top 5 configuraciones:")
print(top5.to_string(index=False))
```

Pista: En GridSearchCV con Pipeline, los parámetros se nombran como "nombre_paso__parametro". El __ doble separa el nombre del paso del parámetro del transformador.

◆ QUIZ RÁPIDO

¿Por qué usar Pipeline previene el data leakage en validación cruzada?

A

Porque Pipeline separa automáticamente train y test en carpetas diferentes

B

Porque `fit()` y `transform()` se aplican solo al fold de training en cada iteración CV, sin ver el fold de test ✓

C

Porque Pipeline aplica los transformadores después del split, no antes

D

Pipeline no previene data leakage, solo organiza el código

Respuesta — B. Cuando `cross_val_score` usa un Pipeline, en cada fold llama a `pipeline.fit(X_train_fold)` (el escalador aprende de ese fold), luego `pipeline.predict(X_test_fold)` (aplica el escalador aprendido). Sin Pipeline, si escalas antes de `cross_val_score`, el escalador ya vio el test fold — eso es data leakage.

◆ QUIZ RÁPIDO

¿Cómo referencias el parámetro 'n_estimators' del paso 'rf' en un GridSearchCV con Pipeline?

A 'n_estimators'

B 'rf.n_estimators'

C 'rf__n_estimators' ✓

D 'RandomForestClassifier__n_estimators'

Respuesta — C. La convención de scikit-learn para Pipelines es 'nombre_paso__parametro' con doble guion bajo. Así GridSearchCV sabe que 'n_estimators' pertenece al paso 'rf' del Pipeline.

✓ MÓDULO COMPLETADO

Los Pipelines son la forma correcta de construir modelos de ML listos para producción. En el módulo final del Libro 2 aplicarás todo lo aprendido en un *proyecto completo*: un sistema de recomendación basado en similitud.

// LIBRO II — TÉCNICAS AVANZADAS · MÓDULO 08

· 50 MIN · CIERRE DEL LIBRO 2

Proyecto: *Recomendador.*

Netflix, Spotify, Amazon — todos usan variantes de esto. Construye un sistema de recomendación desde cero.

Los sistemas de recomendación son uno de los usos más tangibles del ML: sugieren productos, películas, canciones o artículos basándose en el comportamiento previo del usuario o en la similitud entre ítems. En este proyecto construirás dos variantes: basado en contenido (similitud entre ítems) y filtrado colaborativo (basado en usuarios similares).

Similitud por coseno

La **similitud por coseno** mide el ángulo entre dos vectores de características. Si dos películas tienen vectores similares (mismos géneros, actores, etc.), el coseno del ángulo entre ellos estará cerca de 1. Es la base del recomendador basado en contenido.

EJERCICIO 1

· fácil

Recomendador basado en contenido

Construye un recomendador de películas basado en similitud de géneros y ratings. Prueba qué recomienda para diferentes películas.

PYTHON

```
import numpy as np
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import MultiLabelBinarizer

# Dataset de películas con géneros
peliculas = pd.DataFrame({
    'titulo': [
        'Inception', 'Interstellar', 'The Matrix', 'Gravity',
        'The Dark Knight', 'Avatar', 'Avengers', 'Iron Man',
        'Titanic', 'La La Land', 'Whiplash', 'Parasite',
        'Get Out', 'Us', 'A Quiet Place',
    ],
    'generos': [
        ['SciFi', 'Thriller', 'Drama'],
        ['SciFi', 'Drama', 'Adventure'],
        ['SciFi', 'Action', 'Thriller'],
        ['SciFi', 'Thriller', 'Drama'],
        ['Action', 'Thriller', 'Crime'],
        ['SciFi', 'Action', 'Adventure'],
        ['Action', 'SciFi', 'Adventure'],
        ['Action', 'SciFi', 'Comedy'],
        ['Drama', 'Romance', 'History'],
        ['Drama', 'Romance', 'Music'],
        ['Drama', 'Music'],
        ['Drama', 'Thriller', 'Crime'],
        ['Horror', 'Thriller'],
        ['Horror', 'Thriller', 'Drama'],
        ['Horror', 'Thriller'],
    ],
    'rating_promedio': [8.8, 8.6, 8.7, 7.7, 9.0, 7.9, 8.4,
    'año': [2010, 2014, 1999, 2013, 2008, 2009, 2012, 2008,
    })

# Encodear géneros con MultiLabelBinarizer
mlb = MultiLabelBinarizer()
generos_encoded = mlb.fit_transform(peliculas['generos'])
generos_df = pd.DataFrame(generos_encoded, columns=mlb.classes_)
```

```

# Features adicionales: año normalizado y rating
peliculas_features = pd.concat([
    generos_df,
    (peliculas[['año']] - 1990) / 30,
    (peliculas[['rating_promedio']] - 5) / 5,
], axis=1)

# Matriz de similitud coseno
sim_matrix = cosine_similarity(peliculas_features)
sim_df = pd.DataFrame(sim_matrix, index=peliculas['titulo'])

def recomendar(titulo, n=4):
    if titulo not in sim_df.index:
        return []
    similitudes = sim_df[titulo].drop(titulo).sort_values(ascending=False)
    recomendaciones = []
    for t, sim in similitudes.head(n).items():
        idx = peliculas[peliculas['titulo'] == t].index[0]
        recomendaciones.append({
            'titulo': t,
            'similitud': sim,
            'generos': ', '.join(peliculas.loc[idx, 'generos']),
            'rating': peliculas.loc[idx, 'rating_promedio']
        })
    return recomendaciones

# Probar recomendaciones
for pelicula in ['Inception', 'The Dark Knight', 'Get Out']:
    print(f"\n→ Si te gustó '{pelicula}', te recomendamos:")
    for r in recomendar(pelicula, n=3):
        print(f"    {r['titulo'][:20]} (sim={r['similitud']:.2f})")

```

Pista: cosine_similarity devuelve una matriz n×n donde el elemento [i,j] es la similitud entre los ítems i y j. Excluimos la película misma con .drop(titulo) antes de ordenar.

Filtrado colaborativo

El filtrado colaborativo no mira las características de los ítems, sino los patrones de valoración de usuarios similares: "usuarios parecidos a ti también valoraron positivamente X". La técnica clásica usa **factorización de matrices** — descompone la matriz de usuario-ítem en vectores latentes que capturan preferencias.

EJERCICIO 2

· intermedio

Filtrado colaborativo con SVD

Implementa un sistema de recomendación colaborativo con SVD. El modelo aprende preferencias ocultas de usuarios y predice ratings para películas que no han visto.

PYTHON

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import TruncatedSVD
from sklearn.metrics import mean_squared_error

np.random.seed(42)

# Simular 50 usuarios, 20 películas, ratings 1-5 (con mucho ruido)
n_users, n_movies = 50, 20
titulos = [f'Película_{i+1}' for i in range(n_movies)]
usuarios = [f'Usuario_{i+1}' for i in range(n_users)]

# Perfiles latentes: 3 tipos de usuario (acción, drama, sci-fi)
perfil_usuario = np.random.choice(['accion', 'drama', 'scifi'], n_users)
perfil_pelicula = np.random.choice(['accion', 'drama', 'scifi'], n_movies)

# Matriz de ratings con preferencias coherentes + ruido
ratings_true = np.zeros((n_users, n_movies))
for i in range(n_users):
    for j in range(n_movies):
        base = 4.0 if perfil_usuario[i] == perfil_pelicula[j] else 3.5
        ratings_true[i, j] = np.clip(base + np.random.randn(), 1, 5)

# Ocultar el 70% de los ratings (simulando que el usuario no los vio)
mask = np.random.rand(n_users, n_movies) < 0.7
ratings_obs = ratings_true.copy()
ratings_obs[mask] = 0 # 0 = no visto

print(f"Ratings observados: {(~mask).sum()} de {n_users * n_movies}")
print(f"Rating medio real: {ratings_true.mean():.2f}")
print(f"Rating medio obs: {ratings_obs[~mask].mean():.2f}")
print()

# Rellenar nulos con la media del usuario para SVD
ratings_filled = ratings_obs.copy()
for i in range(n_users):
    obs = ratings_obs[i, ratings_obs[i, :] > 0]
    if len(obs) > 0:
```



```

        ratings_filled[i, mask[i]] = obs.mean()

# Factorización con SVD truncado (k=3 factores latentes)
svd = TruncatedSVD(n_components=3, random_state=42)
U = svd.fit_transform(ratings_filled)    # n_users × 3
Sigma = np.diag(svd.singular_values_)    # 3 × 3
Vt = svd.components_                    # 3 × n_movies

ratings_pred = U @ Sigma @ Vt

# Evaluar solo en ratings no observados
y_true = ratings_true[mask]
y_pred = ratings_pred[mask].clip(1, 5)
rmse = mean_squared_error(y_true, y_pred)**0.5
print(f"RMSE en ratings no observados: {rmse:.3f}")

# Generar recomendaciones para un usuario
usuario_idx = 0
print(f"\nRecomendaciones para {usuarios[usuario_idx]} (per
peliculas_no_vistas = np.where(mask[usuario_idx])[0]
scores = ratings_pred[usuario_idx, peliculas_no_vistas]
top_idx = peliculas_no_vistas[np.argsort(scores)[::-1][:5]]
for idx in top_idx:
    print(f"    {titulos[idx]:<15} score={ratings_pred[usuari

# Visualización: matriz de ratings original vs predicha
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
im1 = axes[0].imshow(ratings_true, cmap='YlOrRd', vmin=1, v
axes[0].set_title('Ratings reales'); axes[0].set_xlabel('Pe
plt.colorbar(im1, ax=axes[0])
im2 = axes[1].imshow(ratings_pred.clip(1,5), cmap='YlOrRd',
axes[1].set_title('Ratings predichos por SVD'); axes[1].set
plt.colorbar(im2, ax=axes[1])
plt.tight_layout()
plt.show()

```

Pista: TruncatedSVD descompone la matriz $R \approx U \cdot \Sigma \cdot V^t$ donde U captura los factores latentes de usuarios y Vt los

de películas. Los factores latentes representan "géneros" aprendidos automáticamente.

◆ QUIZ RÁPIDO

¿Cuál es el 'problema del arranque en frío' (cold start) en sistemas de recomendación?

A El sistema tarda mucho en cargar cuando hay muchos usuarios

B No se pueden hacer recomendaciones para usuarios nuevos o ítems nuevos sin historial previo ✓

C El sistema se vuelve menos preciso con el tiempo si no se reentrenan los modelos

D Los ratings del primer día son menos fiables que los posteriores

Respuesta — B. El cold start es el principal desafío de los sistemas de recomendación: para un usuario nuevo (sin historial) o un ítem nuevo (sin valoraciones), no hay datos de los que aprender. Las soluciones incluyen recomendadores basados en contenido para nuevos ítems, o encuestas de onboarding para nuevos usuarios.

◆ QUIZ RÁPIDO

¿En qué se diferencia un recomendador basado en contenido del filtrado colaborativo?

A El basado en contenido es más rápido, el colaborativo es más preciso

B El basado en contenido usa características de los ítems; el colaborativo usa patrones de comportamiento de usuarios similares ✓

C El basado en contenido requiere más datos; el colaborativo funciona con pocos

D Solo hay diferencia en el lenguaje de programación usado

Respuesta — B. Basado en contenido: 'esta película es similar a las que te gustaron según sus características (géneros, actores, etc.)'. Colaborativo: 'usuarios con gustos similares a los tuyos también valoraron esto'. El colaborativo es más poderoso pero necesita muchos datos de interacciones.



¡LIBRO 2 COMPLETADO!

Has dominado las técnicas avanzadas de ML:

- Preprocesamiento: encoding, outliers, datos desbalanceados
- Regularización: Ridge, Lasso, ElasticNet
- SVM con kernels para datos no lineales
- Clustering: K-Means y DBSCAN
- PCA para reducción de dimensiones y visualización
- Selección de features: filter, embedded y wrapper
- Pipelines sin data leakage
- Sistemas de recomendación por contenido y colaborativo

El Libro 3 entra en el deep learning: redes neuronales, CNNs para imágenes, NLP y los transformers que están redefiniendo la IA.



Deep Learning

Redes neuronales, transformers e IA generativa

- 01. Redes neuronales artificiales · 45 min
- 02. Backpropagation y gradiente · 45 min
- 03. Redes convolucionales (CNN) · 50 min
- 04. Redes recurrentes (RNN) · 50 min
- 05. Transformers y atención · 55 min
- 06. Fine-tuning de modelos de lenguaje · 60 min
- 07. Agentes y herramientas · 55 min
- 08. Proyecto final – asistente con contexto · 90 min

// PRÓXIMAMENTE · L3-M1

Redes neuronales artificiales

Este capítulo estará disponible en la siguiente edición del libro.

// en desarrollo · próxima edición

// PRÓXIMAMENTE · L3-M2

Backpropagation y gradiente

Este capítulo estará disponible en la siguiente edición del libro.

// en desarrollo · próxima edición

// PRÓXIMAMENTE · L3-M3

Redes convolucionales (CNN)

Este capítulo estará disponible en la siguiente edición del libro.

// en desarrollo · próxima edición

// PRÓXIMAMENTE · L3-M4

Redes recurrentes (RNN)

Este capítulo estará disponible en la siguiente edición del libro.

// en desarrollo · próxima edición

// PRÓXIMAMENTE · L3-M5

Transformers y atención

Este capítulo estará disponible en la siguiente edición del libro.

// en desarrollo · próxima edición

// PRÓXIMAMENTE · L3-M6

Fine-tuning de modelos de lenguaje

Este capítulo estará disponible en la siguiente edición del libro.

// en desarrollo · próxima edición

// PRÓXIMAMENTE · L3-M7

Agentes y herramientas

Este capítulo estará disponible en la siguiente edición del libro.

// en desarrollo · próxima edición

// PRÓXIMAMENTE · L3-M8

Proyecto final — asistente con contexto

*Este capítulo estará disponible en la siguiente edición
del libro.*

// en desarrollo · próxima edición